# אלגוריתמים חמדניים

תכנון וניתוח אלגוריתמים - תרגול שלישי
סמסטר תשפ"ו א' (אתגר)

המצגת באדיבות אלון קרימגנד

# בעיות אופטימיזציה

- רוב הבעיות שנתעסק בהן בקורס הן **בעיות אופטימיזציה**.
- בבעיות אלו, המטרה הינה למצוא פתרון אופטימלי $opt$ מתוך אוסף של פתרונות אפשריים $X$.

# מהו אלגוריתם חמדן?

- לעיתים קשה להגיד במדויק האם אלגוריתם חמדן או לא.
- באופן כללי, נגיד שאלגוריתם הינו חמדן אם בכל שלב בו, מבין כמה אפשרויות אפשריות, **הוא בוחר את האפשרות הטובה ביותר בעבור קריטריון מסוים** קצר רואי, ללא בחינת או התחשבות בטווח הארוך.
- בדרך כלל קל למצוא אלגוריתמים חמדניים לבעיות אופטימיזציה שונות.
  - אך רק במקרים מסוימים האלגוריתמים החמדניים הינם אידיאליים.
  - במקרים אלו, תצטרכו להוכיח כי הפתרון החמדני הוא הנכון (אופטימלי).

# הוכחת אופטימליות האלגוריתם החמדני

ישנן שתי גישות עיקריות להוכחת נכונות של אלגוריתם חמדן.

1. **טיעון ההשוואה**
   - נשווה בכל שלב ריצה את האלגוריתם החמדן לכל אלגוריתם אחר.
   - נראה כי בכל שלב האלגוריתם החמדן בוחר פתרון לא פחות טוב מהאלגוריתם האחר.

# הוכחת אופטימליות האלגוריתם החמדני

ישנן שתי גישות עיקריות להוכחת נכונות של אלגוריתם חמדן.

1. **טיעון ההשוואה**
   - נשווה בכל שלב ריצה את האלגוריתם החמדן לכל אלגוריתם אחר.
   - נראה כי בכל שלב האלגוריתם החמדן בוחר פתרון לא פחות טוב מהאלגוריתם האחר.
   - בכך נראה כי בכל שלב האלגוריתם החמדן לא פחות טוב מכל אלגוריתם אחר.
   - לכן גם התוצאה הסופית תהיה האופטימלית, כלומר האלגוריתם החמדן אופטימלי כנדרש.

# הוכחת אופטימליות האלגוריתם החמדני

ישנן שתי גישות עיקריות להוכחת נכונות של אלגוריתם חמדן.

1. **טיעון ההשוואה**
   - נשווה בכל שלב ריצה את האלגוריתם החמדן לכל אלגוריתם אחר.
   - נראה כי בכל שלב האלגוריתם החמדן בוחר פתרון לא פחות טוב מהאלגוריתם האחר.
   - בכך נראה כי בכל שלב האלגוריתם החמדן לא פחות טוב מכל אלגוריתם אחר.
   - לכן גם התוצאה הסופית תהיה האופטימלית, כלומר האלגוריתם החמדן אופטימלי כנדרש.
2. **טיעון ההחלפה**
   - נתבונן על פתרון אופטימלי כלשהו.
   - על פתרון זה נבצע החלפות שאינן פוגעות באופטימליותו, עד שנגיע לפתרון שקול שהאלגוריתם החמדן מציע.

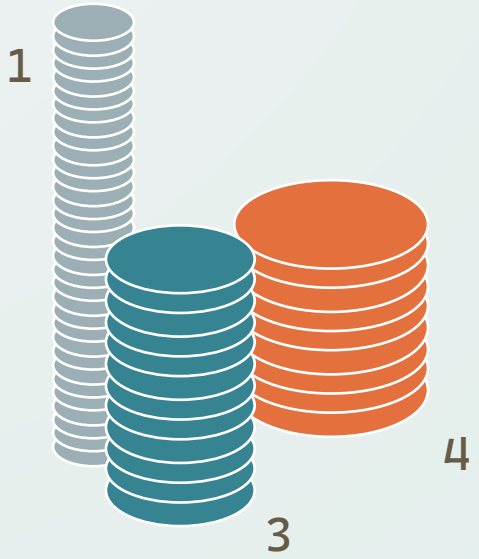
# הוכחת אופטימליות האלגוריתם החמדני

ישנן שתי גישות עיקריות להוכחת נכונות של אלגוריתם חמדן.

1. **טיעון ההשוואה**
   - נשווה בכל שלב ריצה את האלגוריתם החמדן לכל אלגוריתם אחר.
   - נראה כי בכל שלב האלגוריתם החמדן בוחר פתרון לא פחות טוב מהאלגוריתם האחר.
   - בכך נראה כי בכל שלב האלגוריתם החמדן לא פחות טוב מכל אלגוריתם אחר.
   - לכן גם התוצאה הסופית תהיה האופטימלית, כלומר האלגוריתם החמדן אופטימלי כנדרש.
2. **טיעון ההחלפה**
   - נתבונן על פתרון אופטימלי כלשהו.
   - על פתרון זה נבצע החלפות שאינן פוגעות באופטימליותו, עד שנגיע לפתרון שקול שהאלגוריתם החמדן מציע.
   - לכן נראה כי למרות שזהו אינו הפתרון שהאלגוריתם החמדן מציג, הוא שקול לו.
   - כלומר, האלגוריתם החמדן אכן מציע פתרון אופטימלי, שכן הוא שקול לכל פתרון אופטימלי אחר.

# בעיית המטבעות

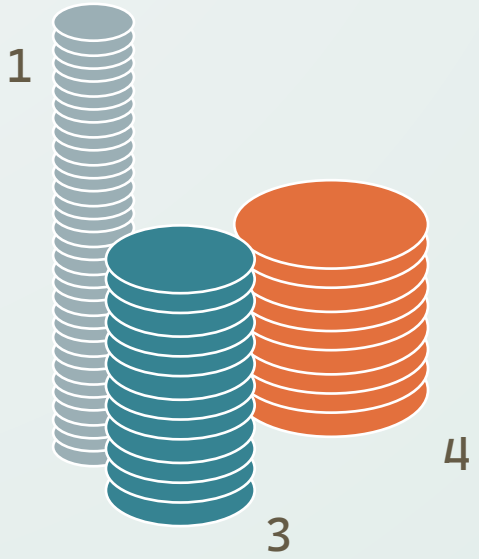
# הגדרה לבעיית המטבעות




$S = \{1,3,4\}$

- נתונה קבוצה של $k$ ערכי מטבעות שלמים שונים $S = \{c_1, c_2, \ldots, c_k\}$
- בנוסף נתון סכום מטרה $n$.
- נרצה להרכיב את הסכום $n$ בעזרת מטבעות מתוך הקבוצה $S$.
- מהו מספר המטבעות המינימלי שנצטרך?
- באילו מטבעות נשתמש כדי לייצג את הסכום בצורה מינימלית?
- האם בכלל אפשר לייצג את הסכום $n$ בעזרת המטבעות?

# הגדרה פורמלית



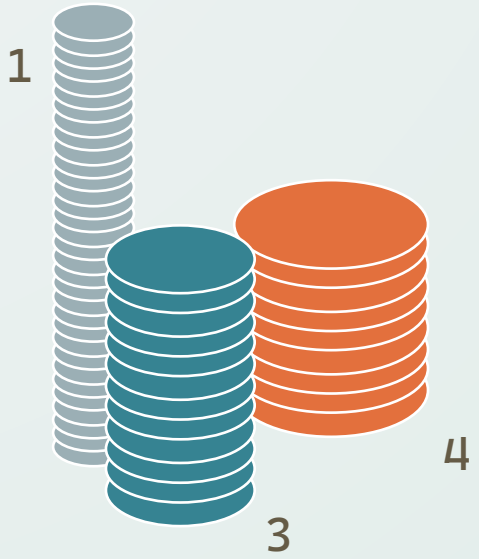

$S = \{1,3,4\}$

נתונה קבוצה $S = \{c_1, c_2, \ldots, c_k\} \subset \mathbb{N}$.

נרצה ש־$\sum \alpha_i$ יהיה מינימלי כאשר $\alpha_i \in \mathbb{N}$ ומתקיים:

$$n = \sum_{i=1}^{k} \alpha_i \cdot c_i$$

# הפתרון החמדני

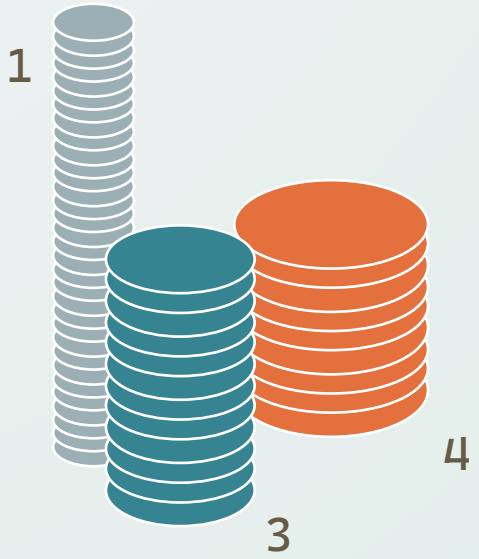



$S = \{1,3,4\}$

- פתרון אפשרי הינו הפתרון החמדני הבא:
- ניקח כל פעם את המטבע הכי גדול שהוספתו אינה חורגת מהסכום
- האם הפתרון יעבוד? כיצד נוכל לדעת אם לא ניתן להרכיב את הסכום $n$?

# הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו־$n = 6$:

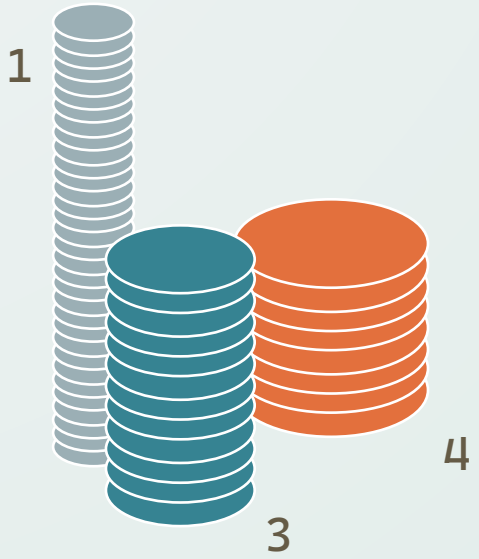



$S = \{1,3,4\}$

נותר לסכום: 6

# הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו־$n = 6$:

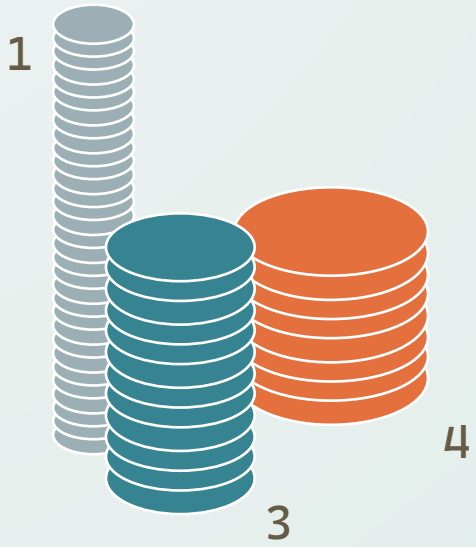



$S = \{1,3,4\}$

נותר לסכום: 2

# הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו־$n = 6$:




$S = \{1,3,4\}$

נותר לסכום: 1

# הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו־$n = 6$:

$S = \{1,3,4\}$

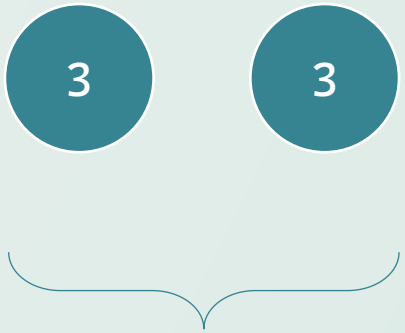
**נותר לסכום: 0**

הפתרון החמדני נתן סכום
בעזרת 3 מטבעות

# הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו־$n = 6$:

$S = \{1,3,4\}$




ישנו פתרון טוב יותר בעזרת
2 מטבעות בלבד!

# מסקנה

$S = \{1,3,4\}$

- בעבור בעיית המטבעות, הפתרון החמדני הטריוויאלי לא עובד!
- מספיק להביא דוגמא שבה הפתרון החמדני לא עובד בכדי לסתור את האופטימליות שלו.
- בתרגול הרביעי נראה פתרון תקין ואופטימלי לבעיית המטבעות, המשתמש בטכניקה הנקראת **תכנון דינאמי**.

# בעיית שיבוץ הסועדים

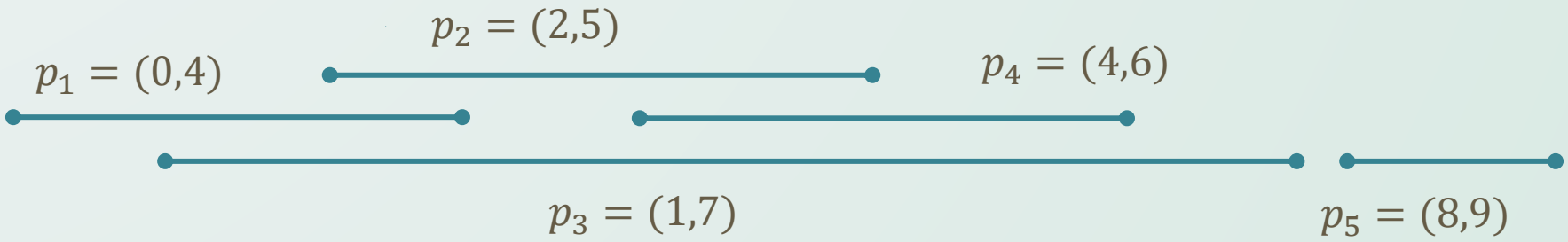
The “Scheduling all Intervals” Problem

# בעיית שיבוץ הסועדים

- אתם בעלים של מסעדה, וברשותכם רשימה של $n$ זוגות **שונים** מהצורה $p_i = (a_i, b_i)$, המייצגים את זמני ההגעה והעזיבה של הסועדים במסעדה.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שישבץ כל סועד $i$ לשולחן מתאים במסעדה, תוך כדי שימוש במספר שולחנות מינימלי**. הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.

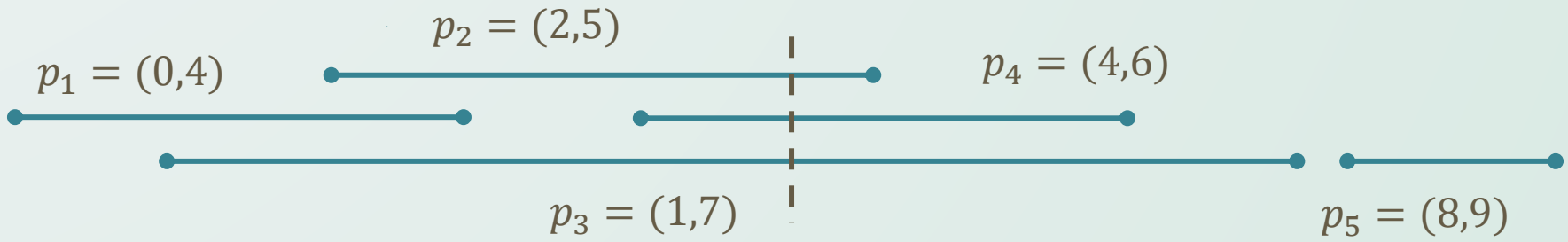
# בעיית שיבוץ הסועדים

- אתם בעלים של מסעדה, וברשותכם רשימה של $n$ זוגות **שונים** מהצורה $p_i = (a_i, b_i)$, המייצגים את זמני ההגעה והעזיבה של הסועדים במסעדה.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שישבץ כל סועד $i$ לשולחן מתאים במסעדה, תוך כדי שימוש במספר שולחנות מינימלי**. הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



# הבחנות בדרך לפתרון

- אם ישנו רגע מסוים שנכלל בתוך $k$ אינטרוולים שונים, כל פתרון ישתמש בלפחות $k$ שולחנות.
  - נאמר **שהעומק** של הרגע הנ"ל הינו $k$.
- נקרא למקסימום העומקים על כל הרגעים השונים בתור **העומק המקסימלי**.
  - העומק המקסימלי הינו חסם תחתון למספר השולחנות בכל פתרון.
  - נרצה לבנות אלגוריתם חמדני המשתמש במספר שולחנות ששווה לעומק המקסימלי – ואז יהיה הוא אופטימלי.



# הפתרון

- נבצע "סימולציה" של המסעדה.
  - בדומה לבעיית הסועדים מהתרגול הראשון, נמיין את כל הנקודות המעניינות.
  - לאחר המיון, נעבור עליהן לפי הסדר.
- בכל פעם שמגיע סועד חדש, נושיב אותו באחד השולחנות הפנויים שכבר "פתחנו".
  - נוכל לשמור רשימה של השולחנות הפנויים.
  - רשימה זו תתחיל ריקה, ובכל פעם שסועד עוזב, נוסיף את השולחן שבו ישב לרשימה.

# הפתרון

- נבצע "סימולציה" של המסעדה.
  - בדומה לבעיית הסועדים מהתרגול הראשון, נמיין את כל הנקודות המעניינות.
  - לאחר המיון, נעבור עליהן לפי הסדר.
- בכל פעם שמגיע סועד חדש, נושיב אותו באחד השולחנות הפנויים שכבר "פתחנו".
  - נוכל לשמור רשימה של השולחנות הפנויים.
  - רשימה זו תתחיל ריקה, ובכל פעם שסועד עוזב, נוסיף את השולחן שבו ישב לרשימה.
- כשמגיע סועד ואף שולחן לא פתוח, נהיה חייבים להוסיף שולחן ולהושיב אותו בו.
- קל לראות שהאלגוריתם מביא תשובה אופטימלית
  - אם האלגוריתם השתמש ב־$x$ שולחנות, הרי שהיה רגע אחד לפחות שבו היו במסעדה $x-1$ שולחנות אך כולם היו תפוסים, כי זהו התנאי היחיד לפתיחת שולחן חדש. ולכן אי אפשר לשבץ בפחות מ־$x$ שולחנות, כנדרש. – טיעון השוואה.
- סיבוכיות הפתרון הינה לינארית + סיבוכיות מיון.

# שיבוץ הסועדים בשולחן בודד

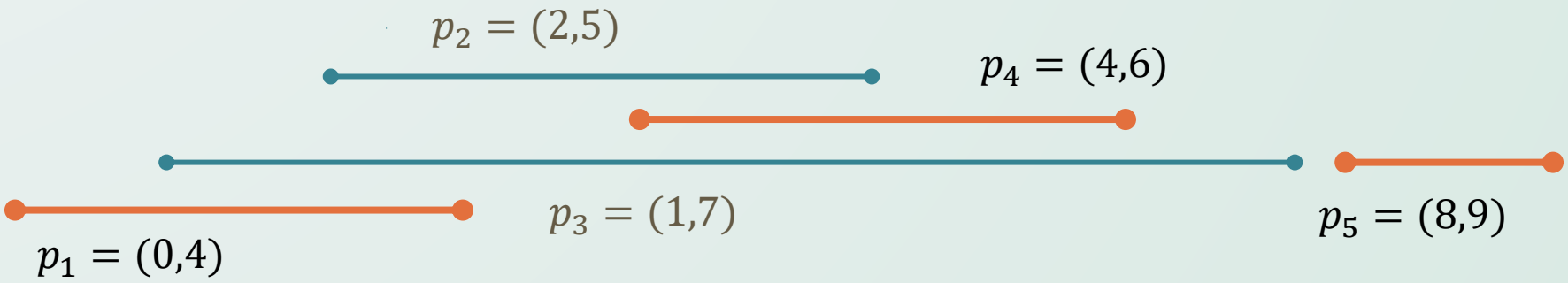
The Interval Scheduling Problem

# בעיית שיבוץ הסועדים בשולחן בודד

- אתם בעלים של מסעדה, וברשותכם רשימה של $n$ זוגות **שונים** מהצורה $p_i = (a_i,b_i)$, המייצגים את זמני ההגעה והעזיבה של הסועדים במסעדה.
- כעת יש ברשותכם **שולחן בודד**, ועליכם לבחור אילו סועדים להושיב ואילו לדחות (בדרכי שלום).
  - כמובן שלא יכולים לשבת שני סועדים בשולחן באותו הזמן. כלומר האינטרוולים של כל הסועדים שבחרתם - זרים.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שיבחר אילו סועדים להושיב, כך שמספר הסועדים שנבחר יהיה מקסימלי.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.

# בעיית שיבוץ הסועדים בשולחן בודד

- אתם בעלים של מסעדה, וברשותכם רשימה של $n$ זוגות **שונים** מהצורה $p_i = (a_i,b_i)$, המייצגים את זמני ההגעה והעזיבה של הסועדים במסעדה.
- כעת יש ברשותכם **שולחן בודד**, ועליכם לבחור אילו סועדים להושיב ואילו לדחות (בדרכי שלום).
  - כמובן שלא יכולים לשבת שני סועדים בשולחן באותו הזמן. כלומר האינטרוולים של כל הסועדים שבחרתם - זרים.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שיבחר אילו סועדים להושיב, כך שמספר הסועדים שנבחר יהיה מקסימלי.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



# ניסיונות

- מה אם ננסה לבחור תמיד את האינטרוול **שמתחיל ראשון**?

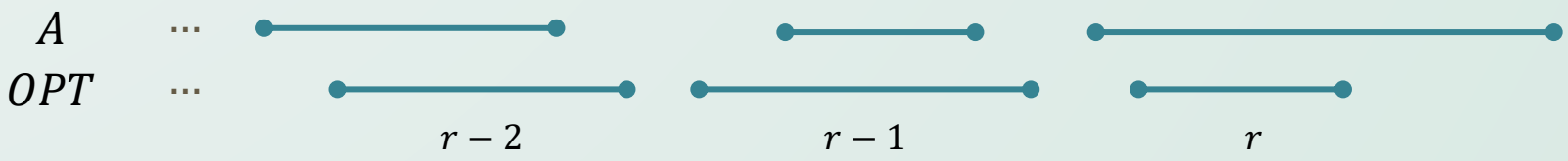
- ומה אם ננסה לבחור את האינטרוולים **הקצרים ביותר** תחילה?

# הפתרון

- נבחר בכל שלב את האינטרוול **שמסתיים ראשון**.
  - כך אנו מוודאים שהשולחן יתפנה במהירות האפשרית, ובו־זמנית אנו משרתים סועד.
- כשנבחר אינטרוול לפתרון, נמחק את כל האינטרוולים שחותכים אותו.
- נריץ את האלגוריתם שוב ושוב, עד שקבוצת האינטרוולים תהיה ריקה.
- אם קבוצת הנקודות נתונה ממוינת, נוכל לבצע את כל הפעולות בזמן לינארי (**איך?**)

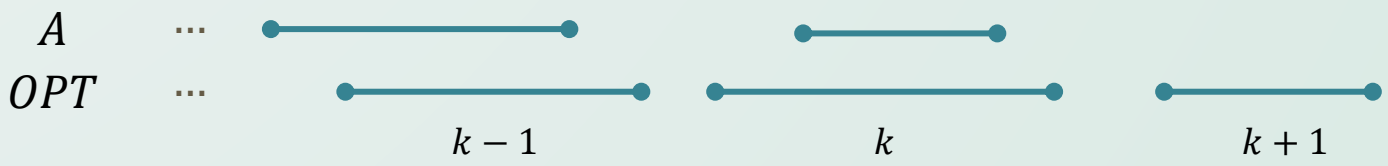
# הוכחת נכונות

- נסמן את הפתרון החמדני בתור $A = \{p_{i_1}, p_{i_2}, \dots p_{i_k}\}$. ניקח פתרון אופטימלי אחר ונסמן את קבוצת האינטרוולים בו בתור $OPT = \{p_{j_1}, p_{j_2}, \dots, p_{j_m}\}$.
- **טענת עזר:** נרצה להוכיח באינדוקציה כי לכל $r \leq k$ מתקיים $b_{i_r} \leq b_{j_r}$.
  - במילים, נרצה להראות כי האינטרוול הראשון בפתרון החמדני מסתיים לא מאוחר יותר מהאינטרוול הראשון בפתרון האופטימלי, וכך גם השני, והשלישי...
- **בסיס האינדוקציה:** כאשר $r = 1$, האלגוריתם החמדני יבחר את האינטרוול המסתיים ראשון, ולכן הוא מקיים $b_{i_1} \leq b_j$ לכל $j$.
- **צעד האינדוקציה:** נניח כי $b_{i_{r-1}} \leq b_{j_{r-1}}$ ונניח בשלילה כי $b_{i_r} > b_{j_r}$. שכנעו את עצמכם שישנה כאן סתירה, והאלגוריתם החמדני היה בוחר באינטרוול $p_{j_r}$ במקום $p_{i_r}$.



# הוכחת נכונות

- מתקיים $|A| = k$ וגם $|OPT| = m$. כדי להוכיח שהפתרון החמדני $A$ אופטימלי, נרצה להראות $m = k$.
- ננית בשלילה כי הפתרון החמדני פחות טוב מהאופטימלי, כלומר ננית כי $m > k$.
- נתבונן כעת על האינטרוול הנוסף של הפתרון האופטימלי.
  - מתקיים $b_{i_k} \leq b_{j_k}$ - האינטרוול ה־$k$ בפתרון החמדני לא מסתיים אחרי אותו האינטרוול בפתרון האופטימלי.
  - מתקיים $b_{j_k} \leq a_{j_{k+1}}$ - האינטרוול ה־$k+1$ בפתרון האופטימלי מתחיל אחרי שהאינטרוול ה־$k$ מסתיים.
  - לכן מתקיים $b_{i_k} \leq a_{j_{k+1}}$, ובעצם הפתרון החמדני יכול לקחת את האינטרוול $p_{j_{k+1}}$.
- אך האלגוריתם החמדני עצר, למרות שהיה צריך לקחת את האינטרוול – סתירה.



# קירוב לכיסוי בקבוצות

Set Cover Approximation

# בעיית הכיסוי בקבוצות

- בהינתן עולם $U$, וסדרה של קבוצות $S_1,S_2, \dots S_m$, המטרה היא למצוא מספר מינימלי של קבוצות $S_i$, כך שהאיחוד שלהן יהיה העולם כולו.

# בעיית הכיסוי בקבוצות

- בהינתן עולם $U$, וסדרה של קבוצות $S_1,S_2, \dots S_m$, המטרה היא למצוא מספר מינימלי של קבוצות $S_i$, כך שהאיחוד שלהן יהיה העולם כולו.
- לדוגמא, עבור העולם $U = \{1,2,3,4,5,6\}$ והקבוצות

$$S_1 = \{1,4,6\},S_2 = \{2,3,6\},S_3 = \{4,5\},S_4 = \{1,3,5\},S_5 = \{1\}$$

- המספר המינימלי יהיה 3, לדוגמא $S_1 \cup S_2 \cup S_3 = U$.

# בעיית הכיסוי בקבוצות

- בהינתן עולם $U$, וסדרה של קבוצות $S_1,S_2, \dots S_m$, המטרה היא למצוא מספר מינימלי של קבוצות $S_i$, כך שהאיחוד שלהן יהיה העולם כולו.
- לדוגמא, עבור העולם $U = \{1,2,3,4,5,6\}$ והקבוצות

$$S_1 = \{1,4,6\},S_2 = \{2,3,6\},S_3 = \{4,5\},S_4 = \{1,3,5\},S_5 = \{1\}$$

- המספר המינימלי יהיה 3, לדוגמא $S_1 \cup S_2 \cup S_3 = U$.
- לצערנו, זו בעיה NP-קשה.
- לכן, ננסה למצוא לה קירוב חמדני.

# בעיית הכיסוי בקבוצות – הצעה לאלגוריתם

- נסתכל על האלגוריתם הבא – בשלב הראשון, ניקח את הקבוצה הכי גדולה. לאחר מכן, בכל שלב ניקח את הקבוצה שמכסה הכי הרבה איברים בעולם שעדיין לא כוסו. נמשיך כך עד שנכסה את כל הקבוצה.

# בעיית הכיסוי בקבוצות – הצעה לאלגוריתם

- נסתכל על האלגוריתם הבא – בשלב הראשון, ניקח את הקבוצה הכי גדולה. לאחר מכן, בכל שלב ניקח את הקבוצה שמכסה הכי הרבה איברים בעולם שעדיין לא כוסו. נמשיך כך עד שנכסה את כל הקבוצה.
- ברור כשמש שנגיע לכיסוי של הקבוצה, אם קיים כזה. נוכיח שאם הכיסוי האופטימלי בגודל $c$, כלומר מכיל $c$ קבוצות, אז הכיסוי החמדני יהיה בגודל $r \leq c \cdot \ln |U|$.

# בעיית הכיסוי בקבוצות – הצעה לאלגוריתם

- נסתכל על האלגוריתם הבא – בשלב הראשון, ניקח את הקבוצה הכי גדולה. לאחר מכן, בכל שלב ניקח את הקבוצה שמכסה הכי הרבה איברים בעולם שעדיין לא כוסו. נמשיך כך עד שנכסה את כל הקבוצה.
- ברור כשמש שנגיע לכיסוי של הקבוצה, אם קיים כזה. נוכיח שאם הכיסוי האופטימלי בגודל $c$, כלומר מכיל $c$ קבוצות, אז הכיסוי החמדני יהיה בגודל $\mathrm{r} \leq c \cdot \ln|U|$. למען האמת נוכיח ש
$$\mathrm{r} - 1 \leq c \cdot \ln|U|$$
על מנת להקל על חישובים מגעילים. לנוחות, נסמן $k = r - 1$.
- **תרגיל** – שכנעו את עצמכם שסיבוכיות האלגוריתם לכל היותר $O(m^2|U|)$ במימוש נאיבי.

# בעיית הכיסוי בקבוצות

- ננית שבשלב הi של האלגוריתם, נשארו $x_i$ איברים לא מכוסים. כמובן $x_0 = |U|$.

# בעיית הכיסוי בקבוצות

- נניח שבשלב ה$i$ של האלגוריתם, נשארו $x_i$ איברים לא מכוסים. כמובן $x_0 = |U|$.
- **טענה** – בשלב ה$i$, יכוסו לפחות $x_i/c$ איברים. אנחנו יודעים שיש $c$ קבוצות שיכולות לכסות את האיברים שנשארו, ולכן לפי שובך היונים, קבוצה אחת לפחות תכיל לפחות $x_i/c$ איברים. כיוון שקיימת קבוצה עם הכמות הזו של האיברים, בפרט הקבוצה המקסימלית תכיל לפחות $x_i/c$ איברים.

# בעיית הכיסוי בקבוצות

- ננית שבשלב ה$i$ של האלגוריתם, נשארו $x_i$ איברים לא מכוסים. כמובן $x_0 = |U|$.
- **טענה** – בשלב ה$i$, יכוסו לפחות $x_i/c$ איברים. אנחנו יודעים שיש $c$ קבוצות שיכולות לכסות את האיברים שנשארו, ולכן לפי שובך היונים, קבוצה אחת לפחות תכיל לפחות $x_i/c$ איברים. כיוון שקיימת קבוצה עם הכמות הזו של האיברים, בפרט הקבוצה המקסימלית תכיל לפחות $x_i/c$ איברים.
- לכן, נוכל לטעון ש:

$$x_{i+1} \leq x_i - \frac{x_i}{c} = x_i\left(1 - \frac{1}{c}\right)$$

# בעיית הכיסוי בקבוצות

- נניח שבשלב ה$i$ של האלגוריתם, נשארו $x_i$ איברים לא מכוסים. כמובן $x_0 = |U|$.
- **טענה** – בשלב ה$i$, יכוסו לפחות $x_i/c$ איברים. אנחנו יודעים שיש $c$ קבוצות שיכולות לכסות את האיברים שנשארו, ולכן לפי שובך היונים, קבוצה אחת לפחות תכיל לפחות $x_i/c$ איברים. כיוון שקיימת קבוצה עם הכמות הזו של האיברים, בפרט הקבוצה המקסימלית תכיל לפחות $x_i/c$ איברים.
- לכן, נוכל לטעון ש:

$$x_{i+1} \leq x_i - \frac{x_i}{c} = x_i\left(1 - \frac{1}{c}\right)$$

- האיטרציה האחת לפני האחרונה הייתה האיטרציה ה-$k$ (**למה?**). אזי נדע ש:

$$x_k \leq x_0\left(1 - \frac{1}{c}\right)^k = |U|\left(1 - \frac{1}{c}\right)^k$$

# בעיית הכיסוי בקבוצות

- מצד שני, לקחנו את k להיות האיטרציה האחת לפני אחרונה. כלומר היה עוד לפחות איבר אחד לכסות. על כן, נקבל

$$1 \leq x_k \leq |U|\left(1-\frac{1}{c}\right)^k = |U|\left(\left(1-\frac{1}{c}\right)^c\right)^{\frac{k}{c}}$$

# בעיית הכיסוי בקבוצות

- מצד שני, לקחנו את k להיות האיטרציה האחת לפני אחרונה. כלומר היה עוד לפחות איבר אחד לכסות. על כן, נקבל

$$1 \le x_k \le |U|\left(1-\frac{1}{c}\right)^k = |U|\left(\left(1-\frac{1}{c}\right)^c\right)^{\frac{k}{c}}$$

- נשתמש בעובדה ש $\left(1-\frac{1}{a}\right)^a \le \frac{1}{e}$ ונקבל ש:

$$1 \le |U|\left(\frac{1}{e}\right)^{\frac{k}{c}} \Rightarrow e^{\frac{k}{c}} \le |U| \Rightarrow \boldsymbol{k \le c \cdot ln\,|U|}$$

# בעיית הכיסוי בקבוצות

- מצד שני, לקחנו את k להיות האיטרציה האחת לפני אחרונה. כלומר היה עוד לפחות איבר אחד לכסות. על כן, נקבל

$$1 \le x_k \le |U|\left(1-\frac{1}{c}\right)^k = |U|\left(\left(1-\frac{1}{c}\right)^c\right)^{\frac{k}{c}}$$

- נשתמש בעובדה ש $\left(1-\frac{1}{a}\right)^a \le \frac{1}{e}$ ונקבל ש:

$$1 \le |U|\left(\frac{1}{e}\right)^{\frac{k}{c}} \Rightarrow e^{\frac{k}{c}} \le |U| \Rightarrow \boldsymbol{k \le c \cdot ln\,|U|}$$

- כיוון ש $r = k+1$, הראנו (עד כדי $\pm 1$)

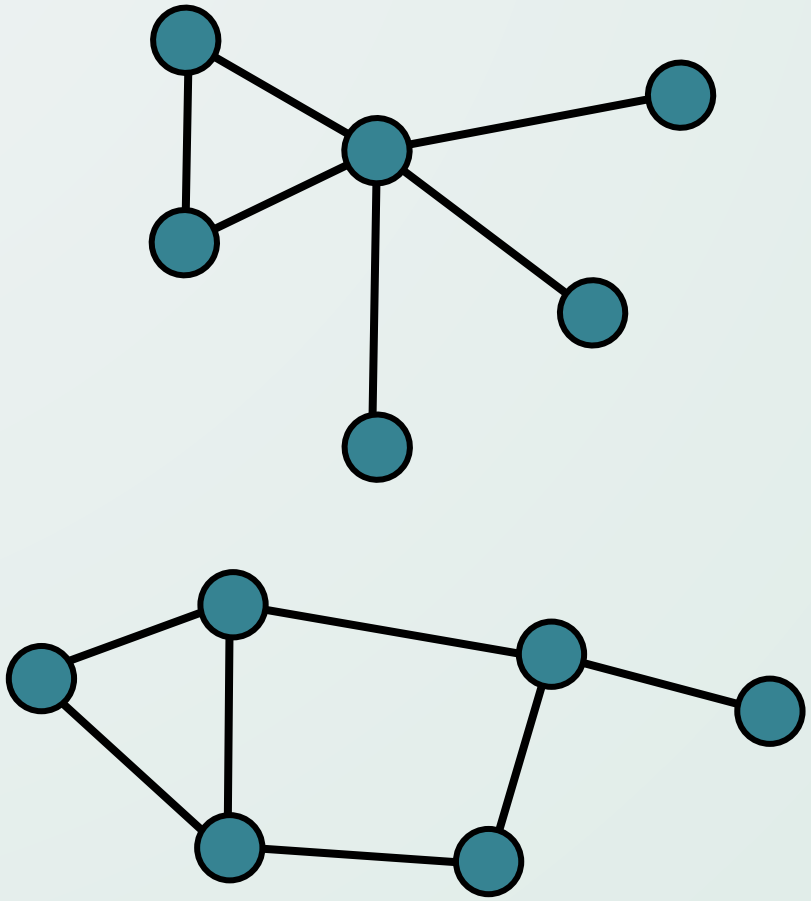
$$r \le c \cdot \ln|U|$$

ואכן השגנו קירוב $\ln|U|$ לאופטימלי.

# כיסוי מינימלי בצמתים על עצים

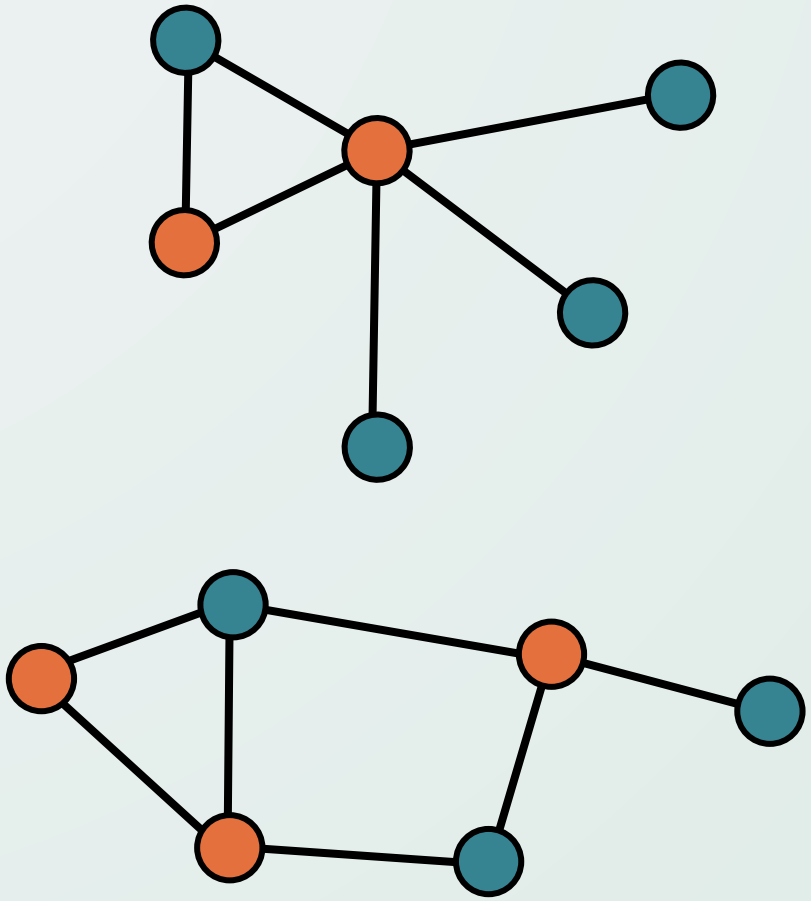
Minimum Vertex Cover on Trees

# בעיית כיסוי מינימלי בצמתים על עצים



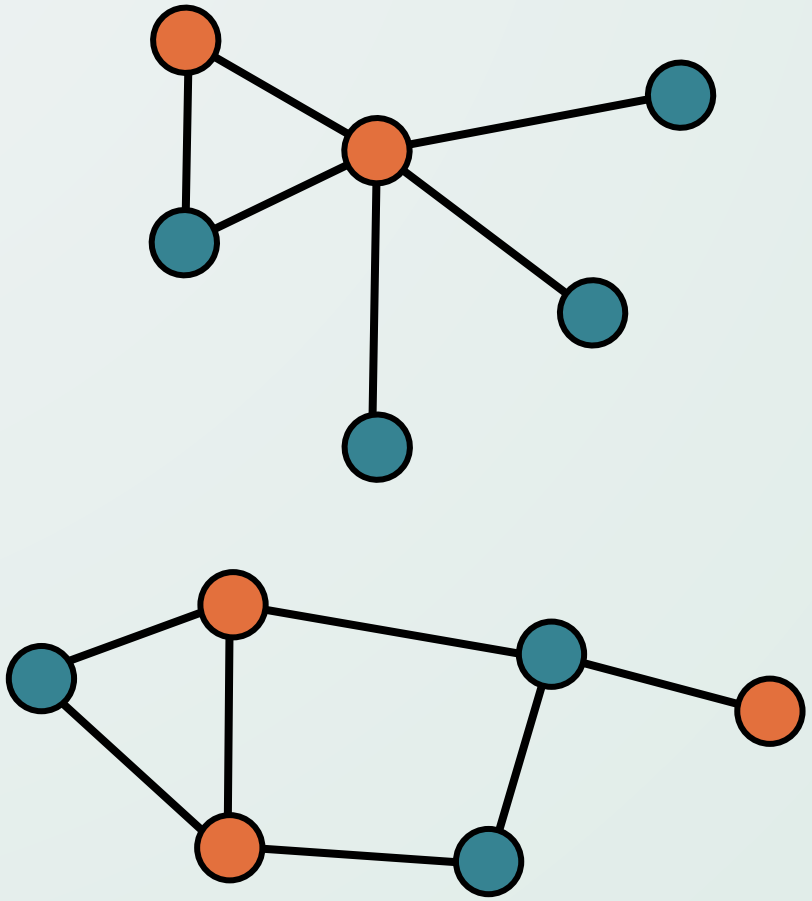
- בהינתן גרף לא מכוון $G = (V,E)$, כיסוי בצמתים של $G$ הוא קבוצת צמתים $\tilde{V} \subseteq V$ כך שלכל קשת $(v,u) \in E$ או ש־$v \in \tilde{V}$ או ש־$u \in \tilde{V}$ (או שניהם).
- כיסוי מינימלי בצמתים של $G$ היא קבוצה מינימלית של צמתים שהיא כיסוי על $G$.
- על גרף כללי זוהי בעיה $\mathrm{NP-Complete}$ - היא קשה. תלמדו על זה בהרחבה בקורס מודלים חישוביים. (או לחילופין – מבחן של יורי סמסטר א 2023)
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שבהינתן עץ, ימצא את הכיסוי המינימלי בו.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.

# בעיית כיסוי מינימלי בצמתים על עצים



- בהינתן גרף לא מכוון $G = (V,E)$, כיסוי בצמתים של $G$ הוא קבוצת צמתים $\tilde{V} \subseteq V$ כך שלכל קשת $(v,u) \in E$ או ש־$v \in \tilde{V}$ או ש־$u \in \tilde{V}$ (או שניהם).
- כיסוי מינימלי בצמתים של $G$ היא קבוצה מינימלית של צמתים שהיא כיסוי על $G$.
- על גרף כללי זוהי בעיה $\mathrm{NP-Complete}$ - היא קשה. תלמדו על זה בהרחבה בקורס מודלים חישוביים. (או לחילופין – מבחן של יורי סמסטר א 2023)
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שבהינתן עץ, ימצא את הכיסוי המינימלי בו.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.

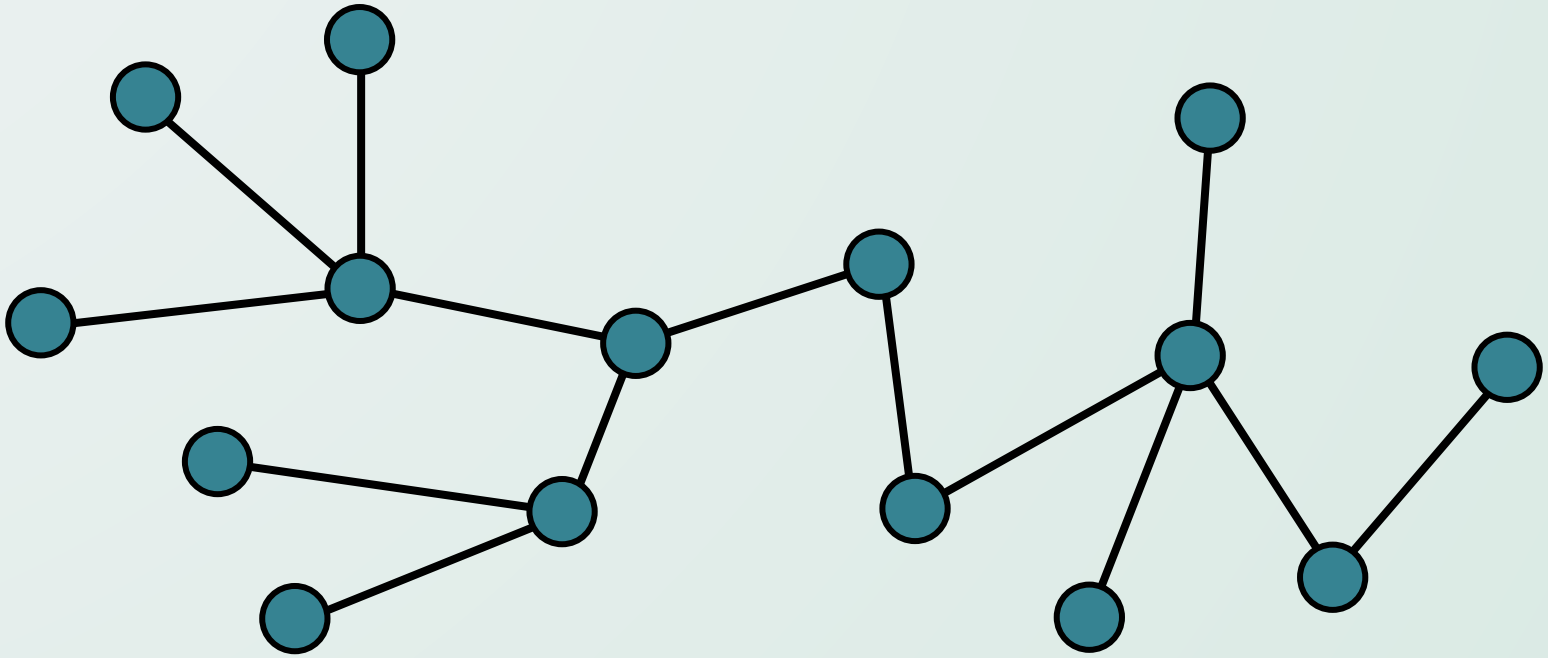
# בעיית כיסוי מינימלי בצמתים על עצים



- בהינתן גרף לא מכוון $G = (V,E)$, כיסוי בצמתים של $G$ הוא קבוצת צמתים $\tilde{V} \subseteq V$ כך שלכל קשת $(v,u) \in E$ או ש־$v \in \tilde{V}$ או ש־$u \in \tilde{V}$ (או שניהם).
- כיסוי מינימלי בצמתים של $G$ היא קבוצה מינימלית של צמתים שהיא כיסוי על $G$.
- על גרף כללי זוהי בעיה $\mathrm{NP-Complete}$ - היא קשה. תלמדו על זה בהרחבה בקורס מודלים חישוביים. (או לחילופין – מבחן של יורי סמסטר א 2023)
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שבהינתן עץ, ימצא את הכיסוי המינימלי בו.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.

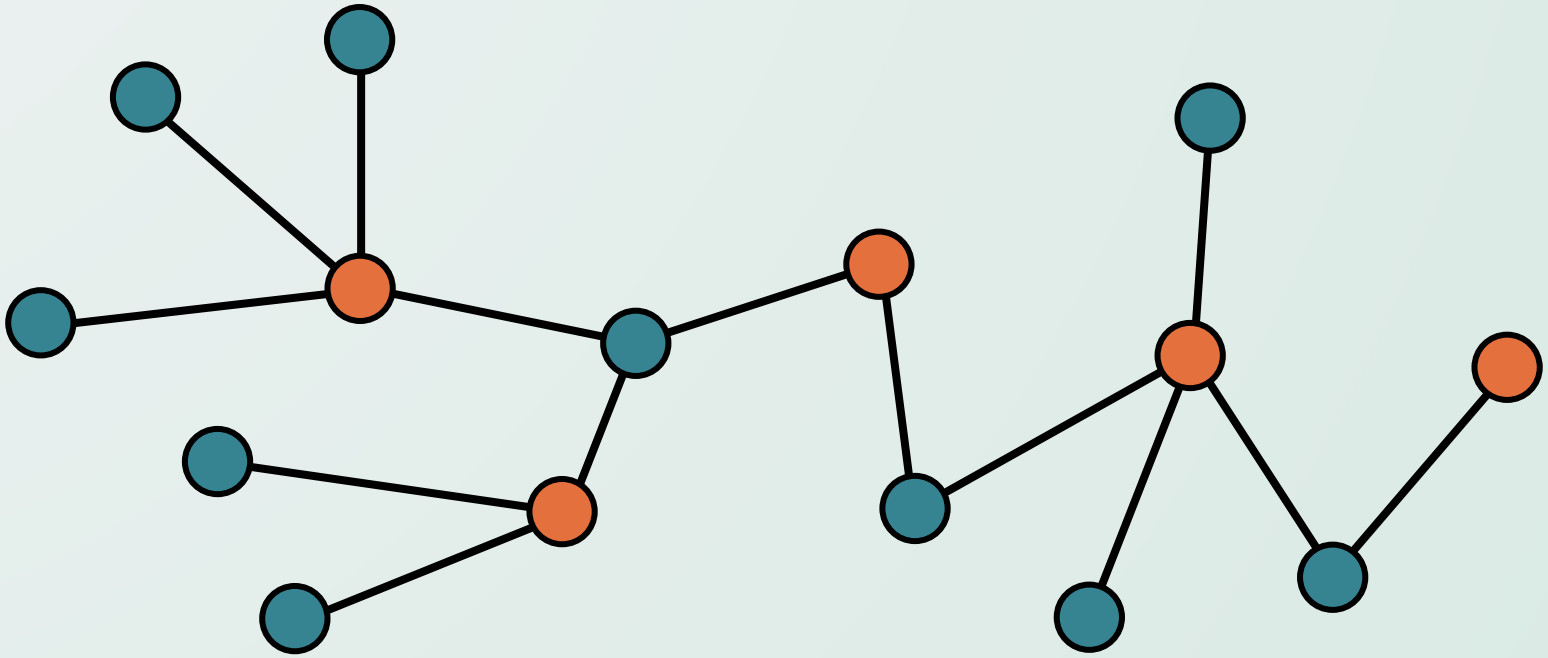
# הבחנות בדרך לפתרון

- קיים כיסוי מינימלי בצמתים על עצים, שבו אף עלה לא בכיסוי.
- **נשתמש בטיעון ההחלפה:** נחליף כל עלה שבכיסוי על ידי ההורה שלו. מספר הצמתים בכיסוי יישאר זהה במקרה הגרוע, אך אם אחד ההורים כבר היה בכיסוי, מספר הצמתים בכיסוי החדש אף יקטן.



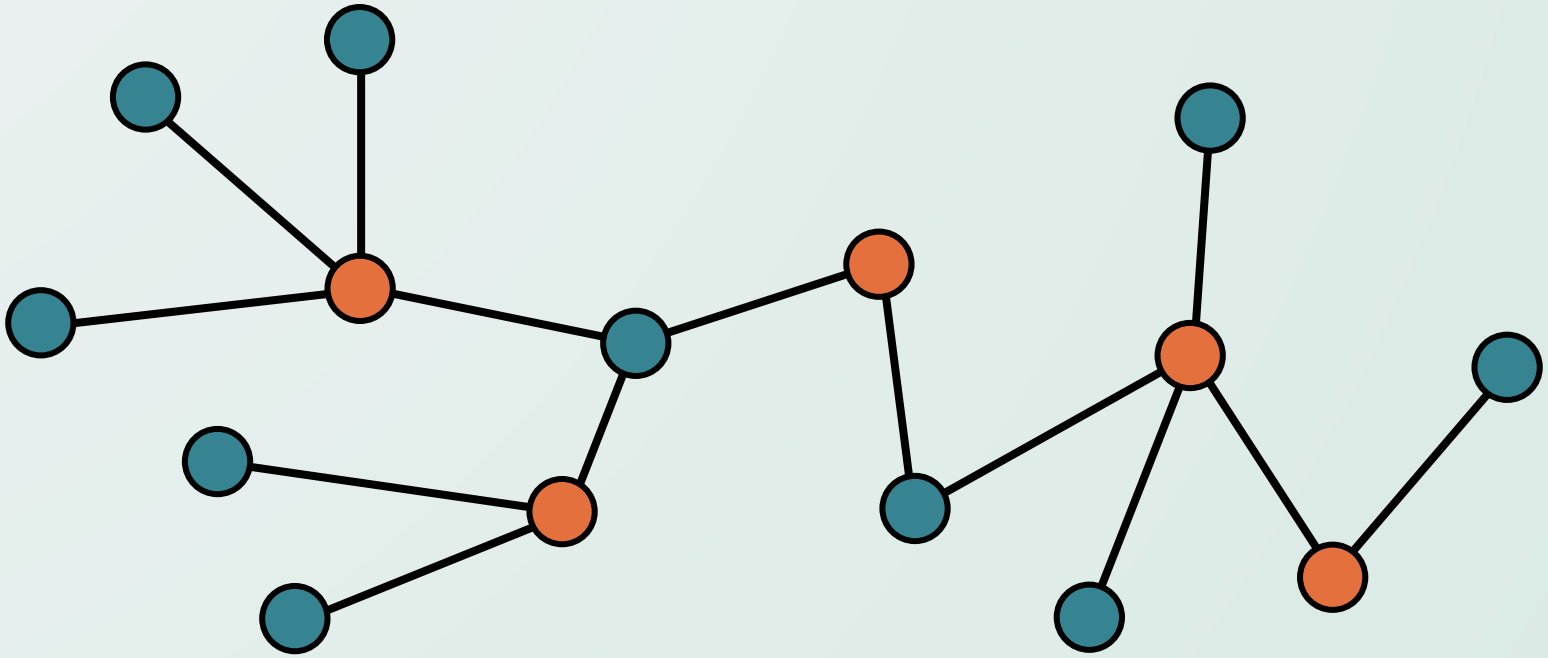
# הבחנות בדרך לפתרון

- קיים כיסוי מינימלי בצמתים על עצים, שבו אף עלה לא בכיסוי.
- **נשתמש בטיעון ההחלפה:** נחליף כל עלה שבכיסוי על ידי ההורה שלו. מספר הצמתים בכיסוי יישאר זהה במקרה הגרוע, אך אם אחד ההורים כבר היה בכיסוי, מספר הצמתים בכיסוי החדש אף יקטן.



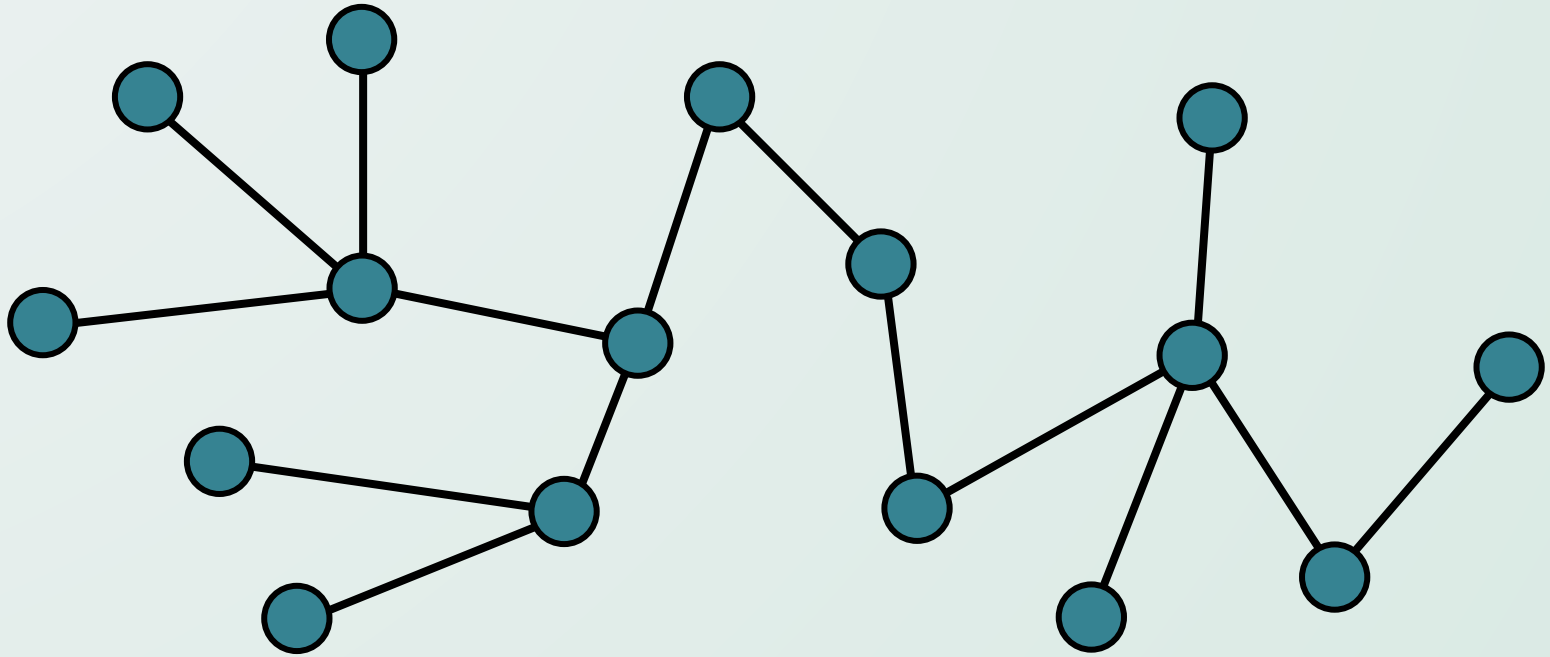
# הבחנות בדרך לפתרון

- קיים כיסוי מינימלי בצמתים על עצים, שבו אף עלה לא בכיסוי.
- **נשתמש בטיעון ההחלפה:** נחליף כל עלה שבכיסוי על ידי ההורה שלו. מספר הצמתים בכיסוי יישאר זהה במקרה הגרוע, אך אם אחד ההורים כבר היה בכיסוי, מספר הצמתים בכיסוי החדש אף יקטן.



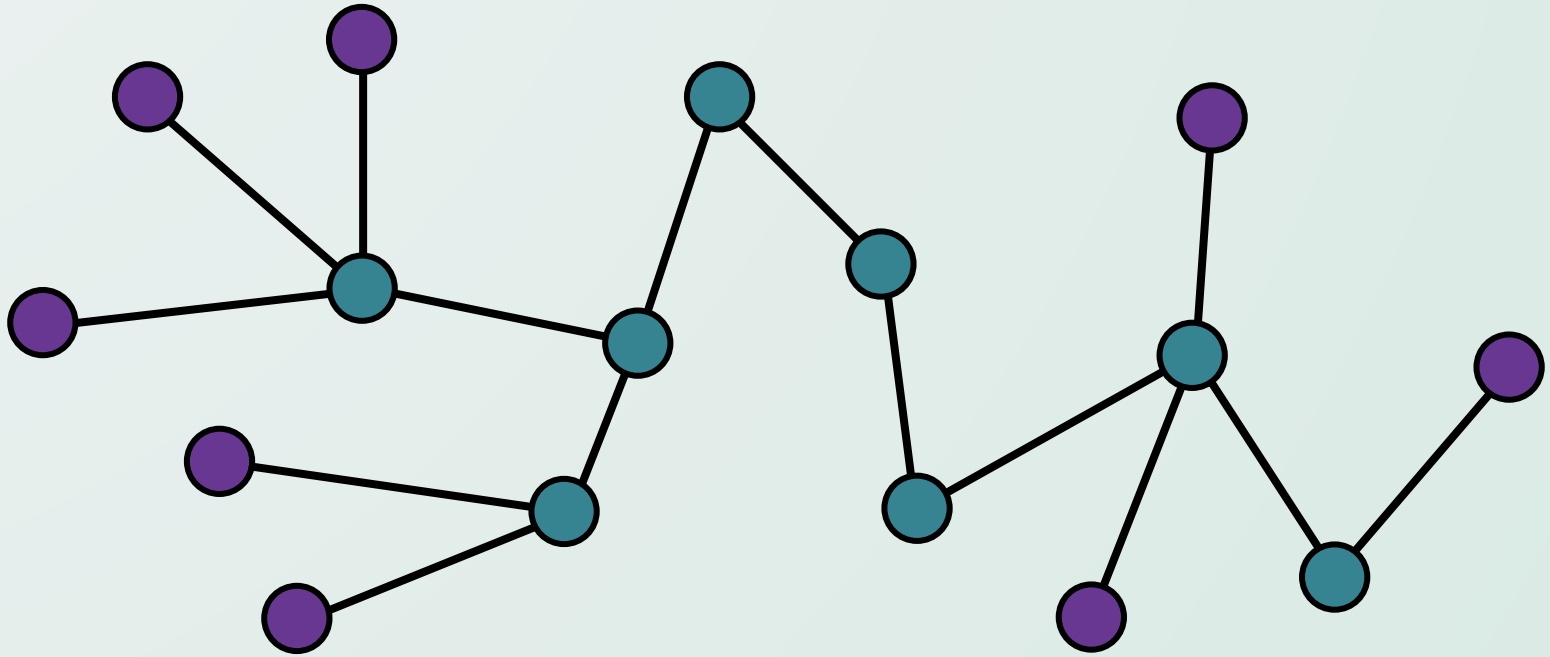
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



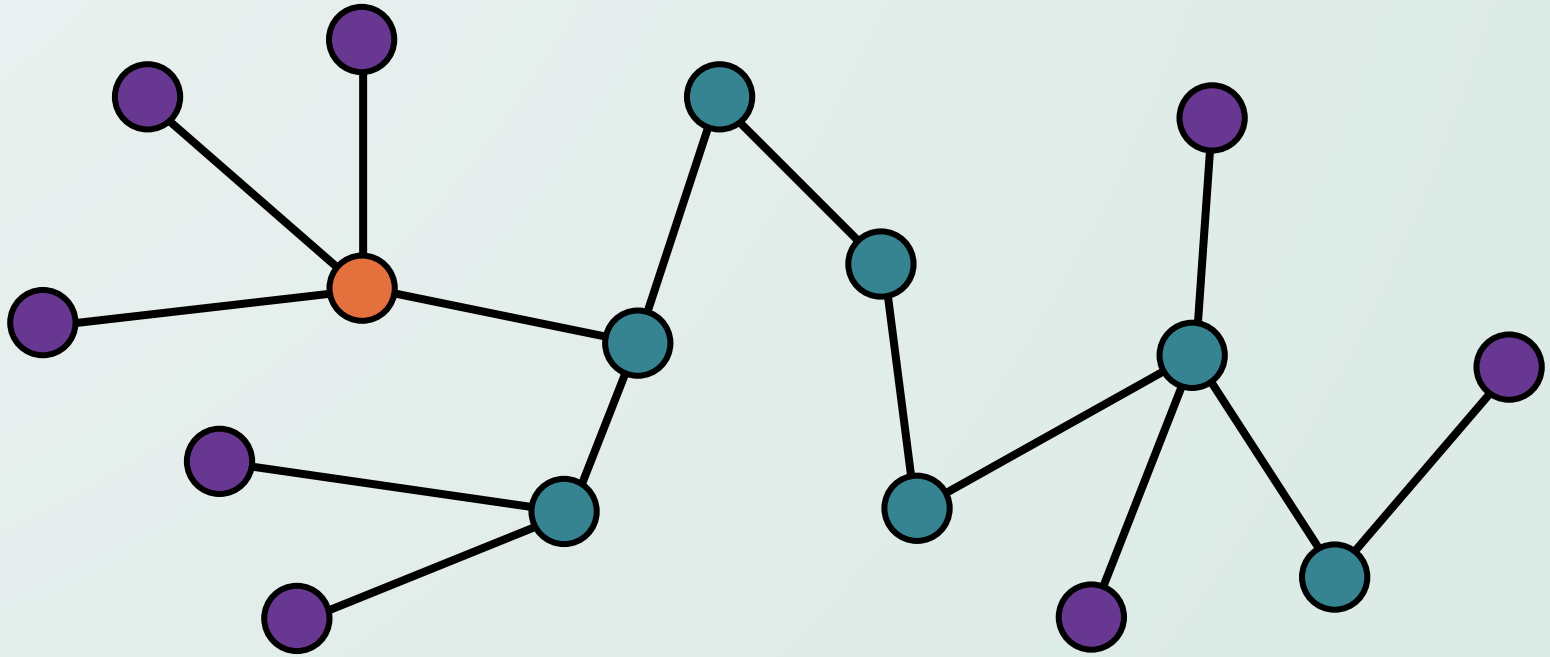
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



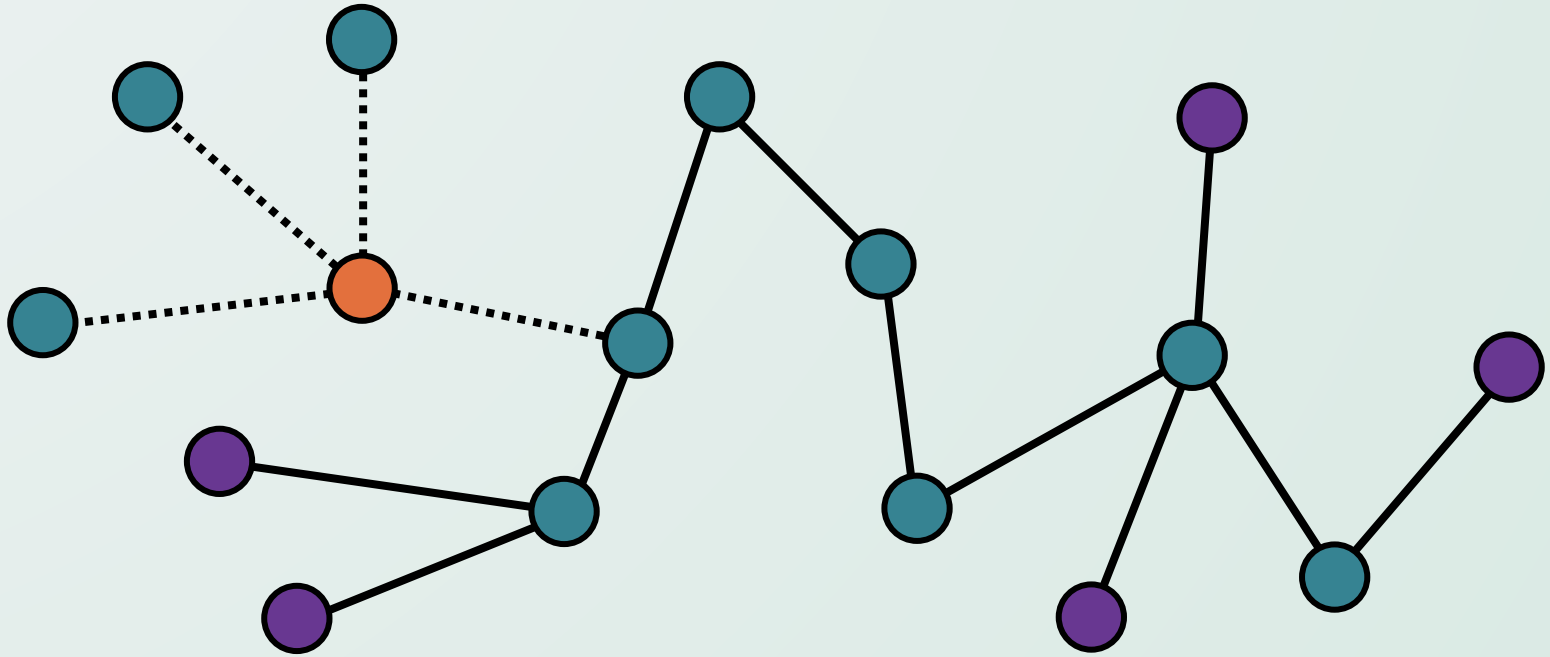
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



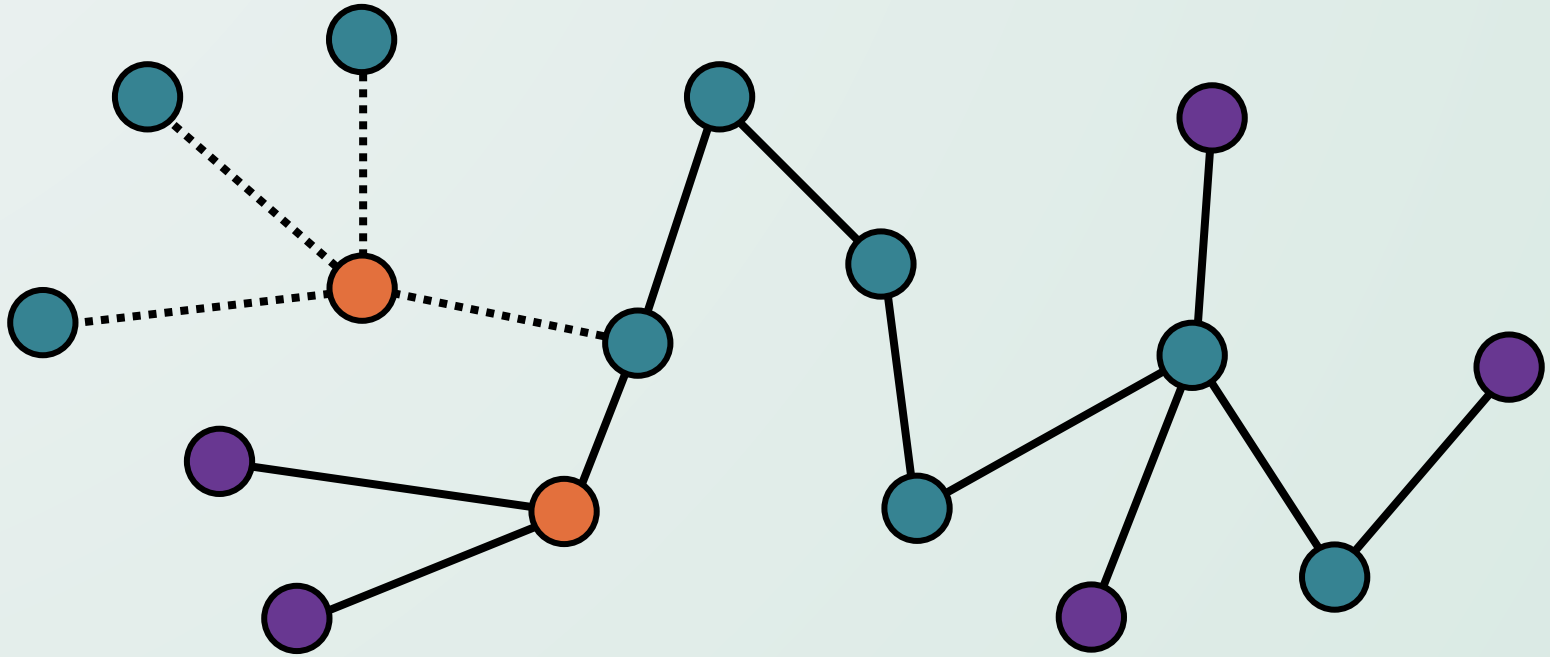
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



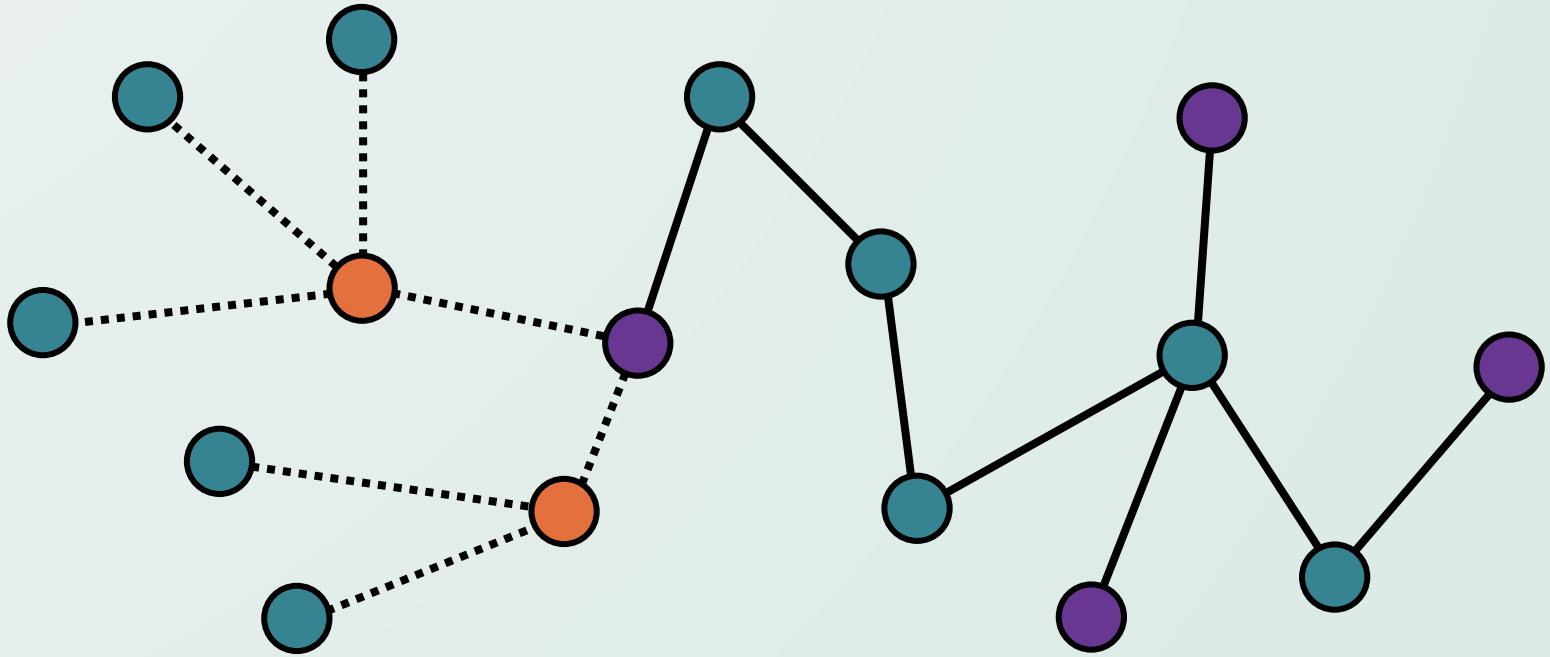
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



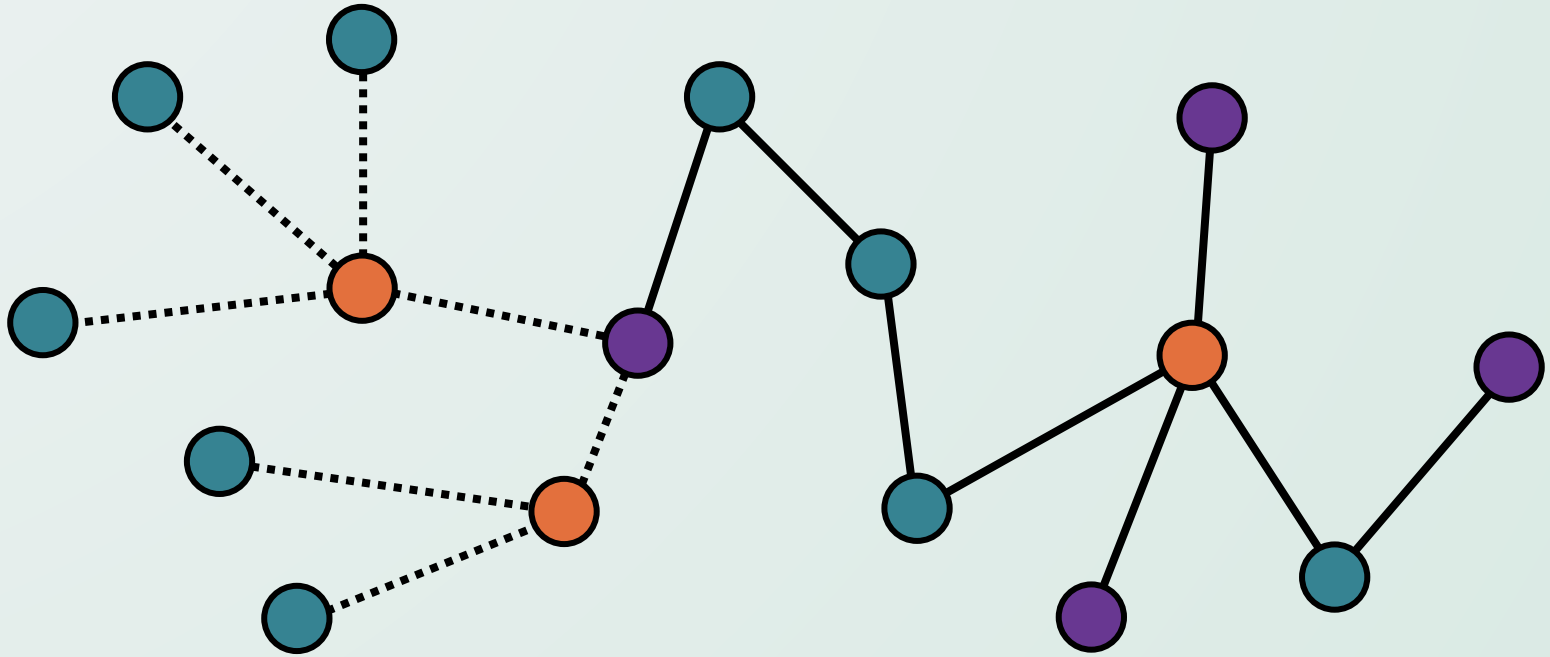
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



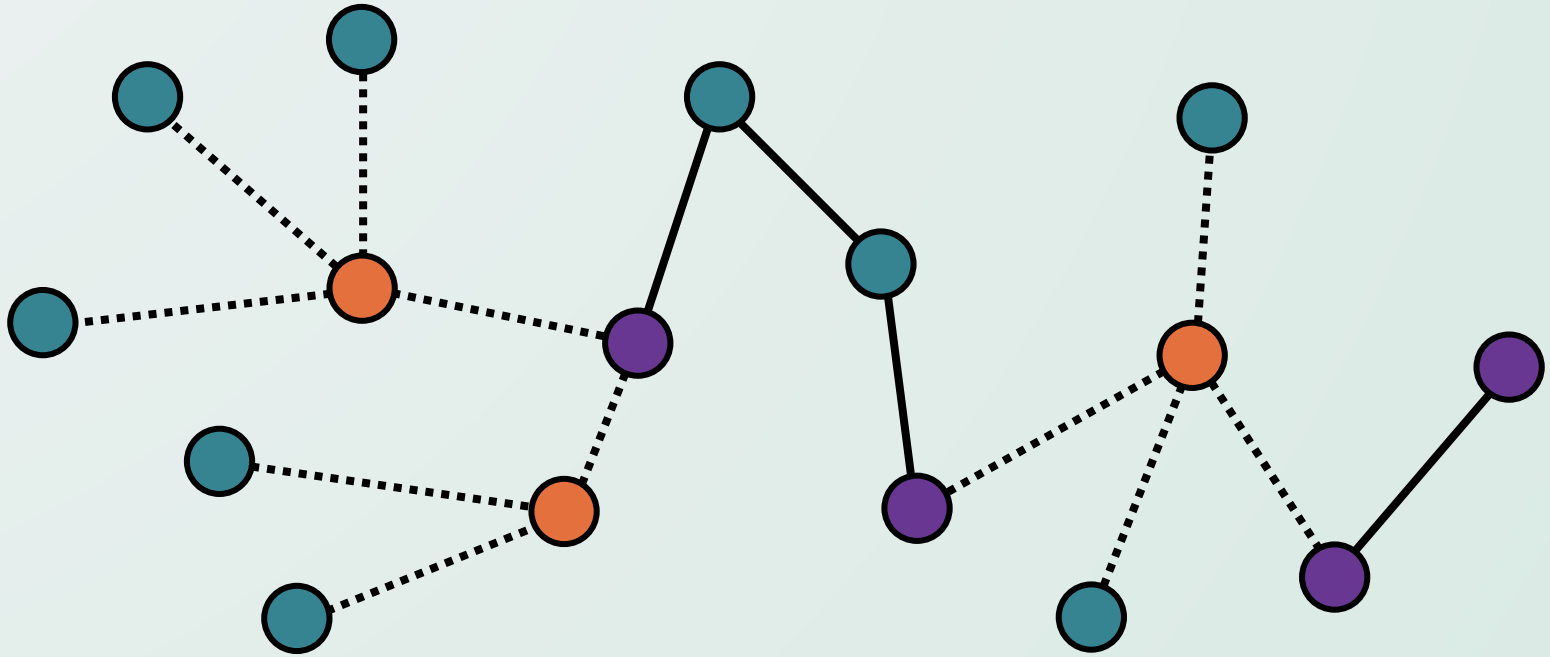
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



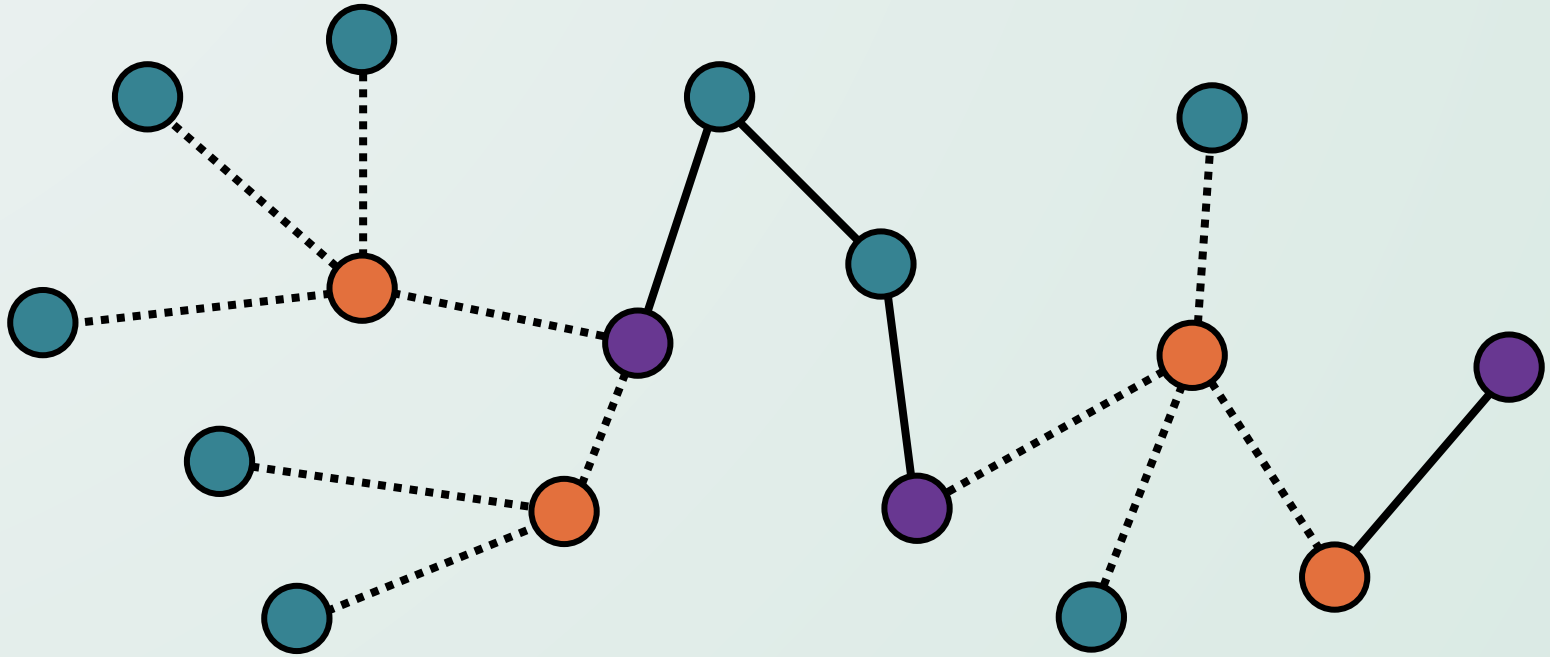
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



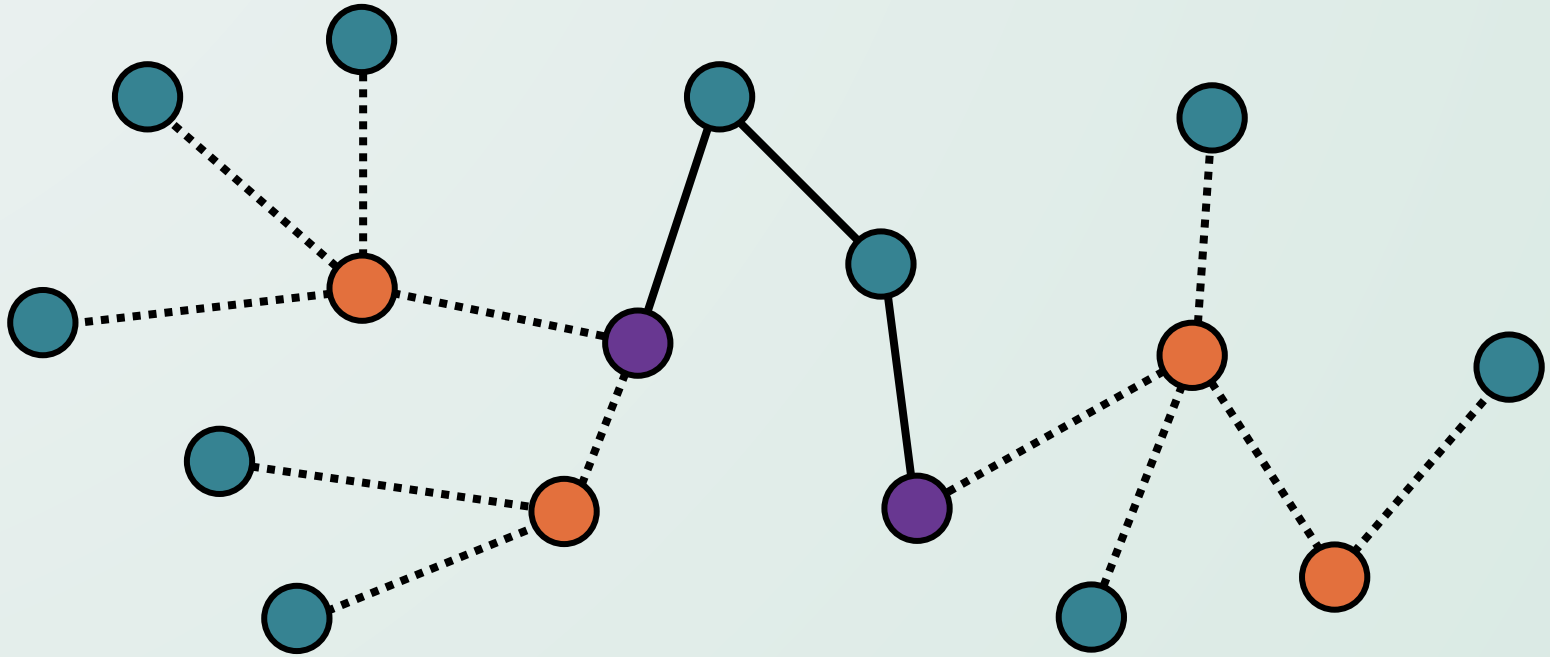
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



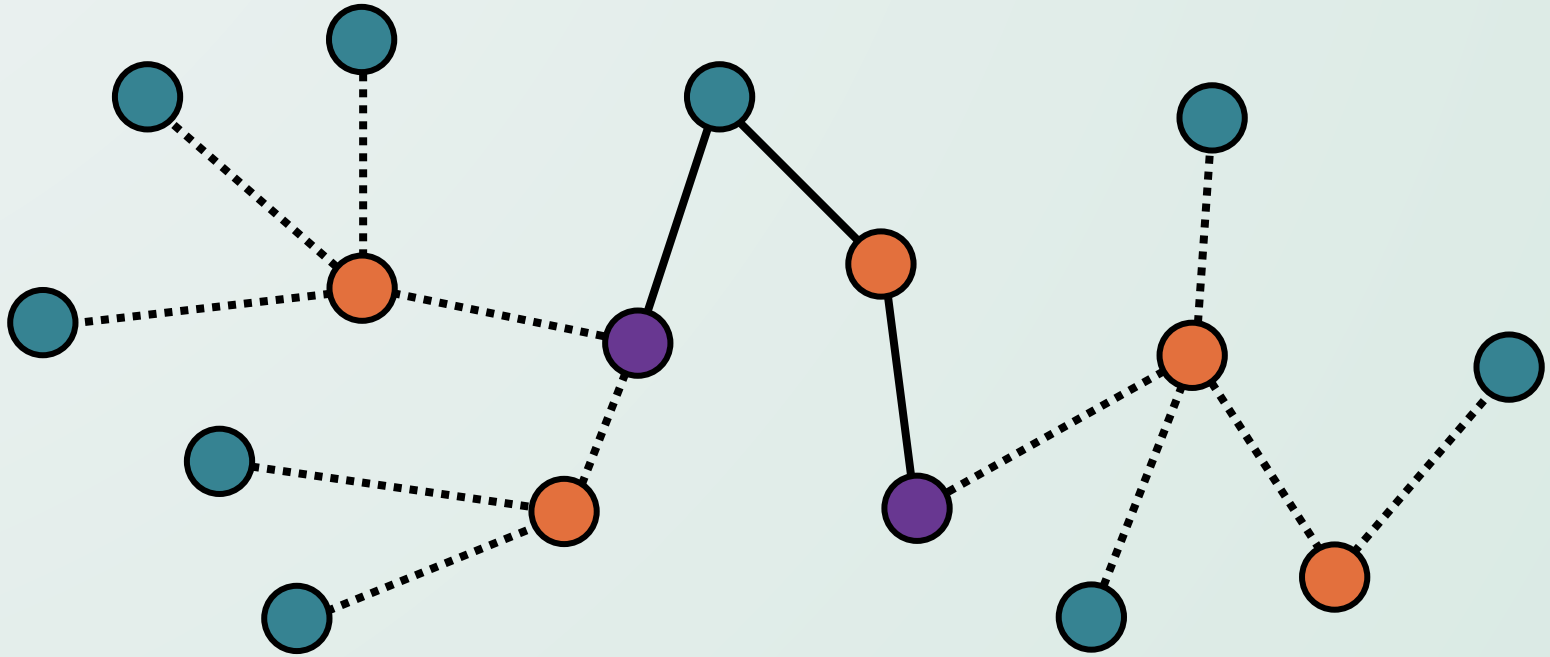
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



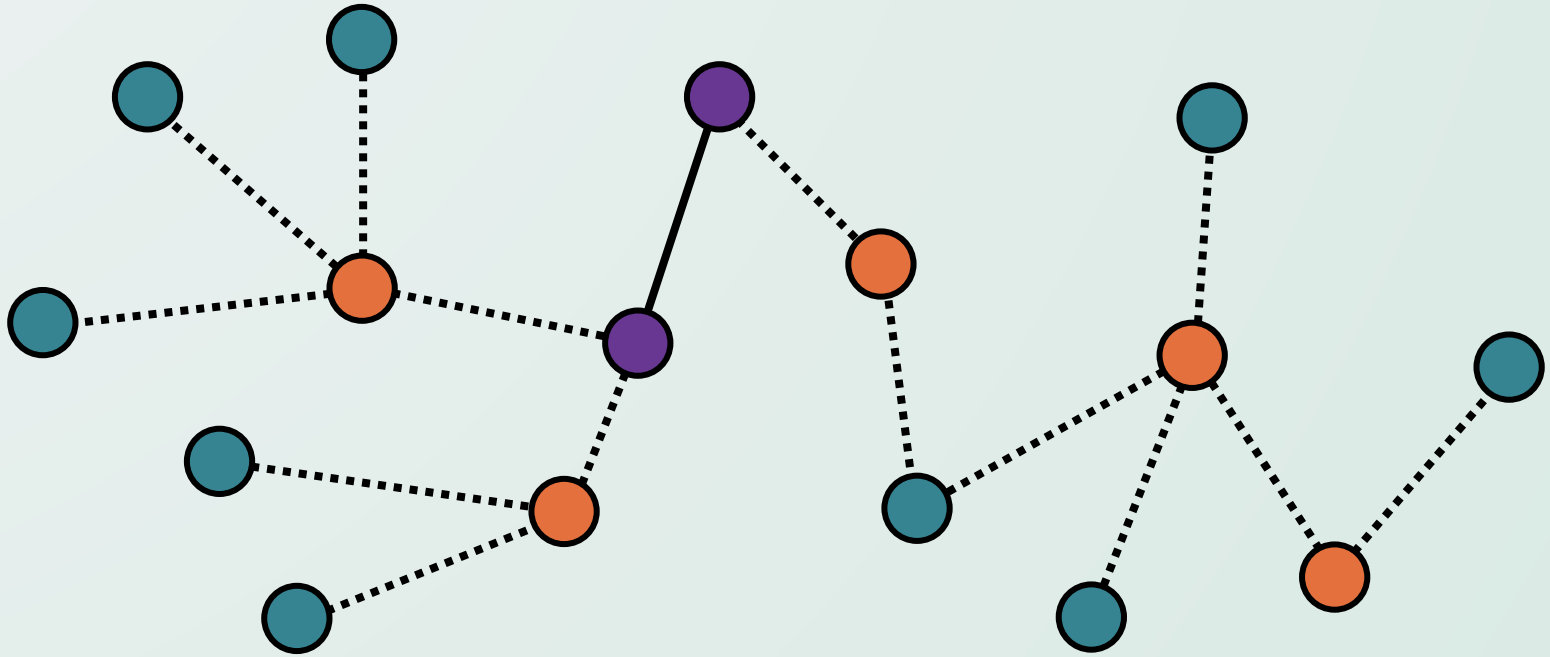
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



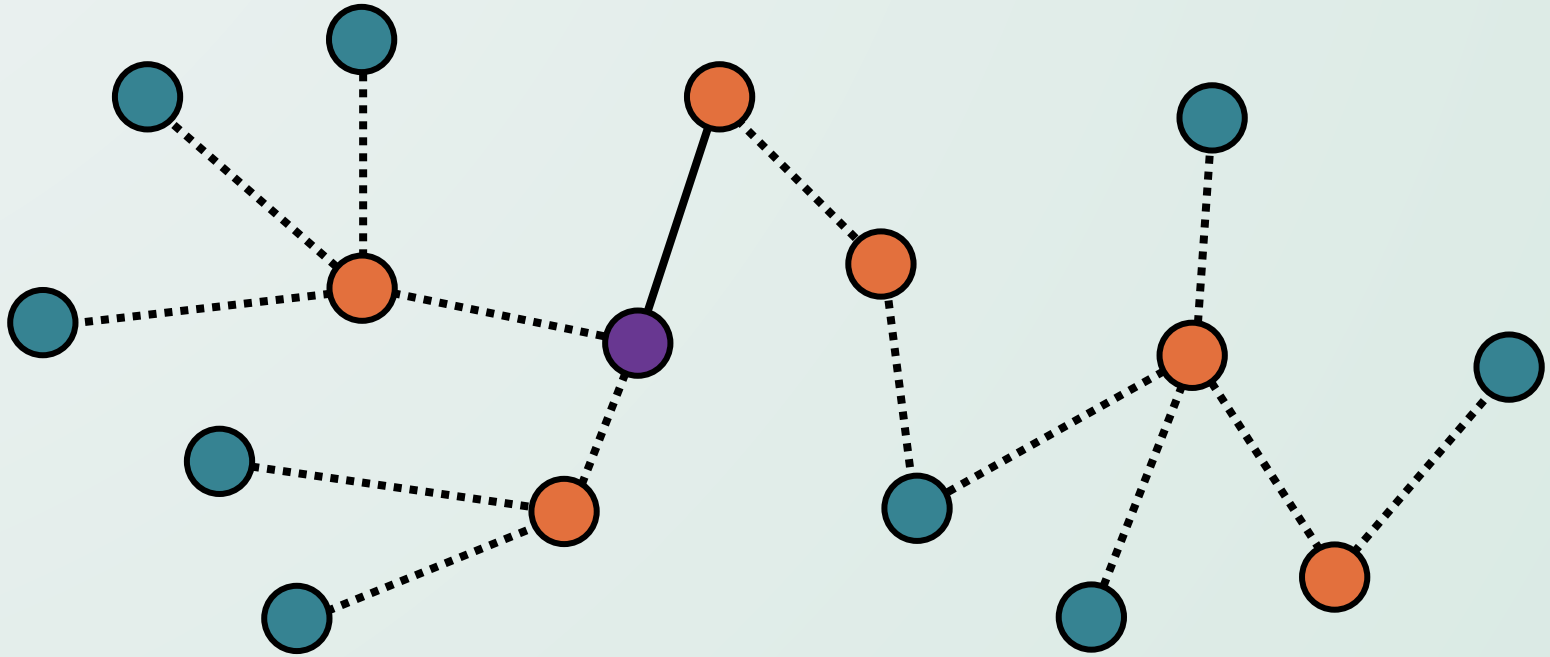
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



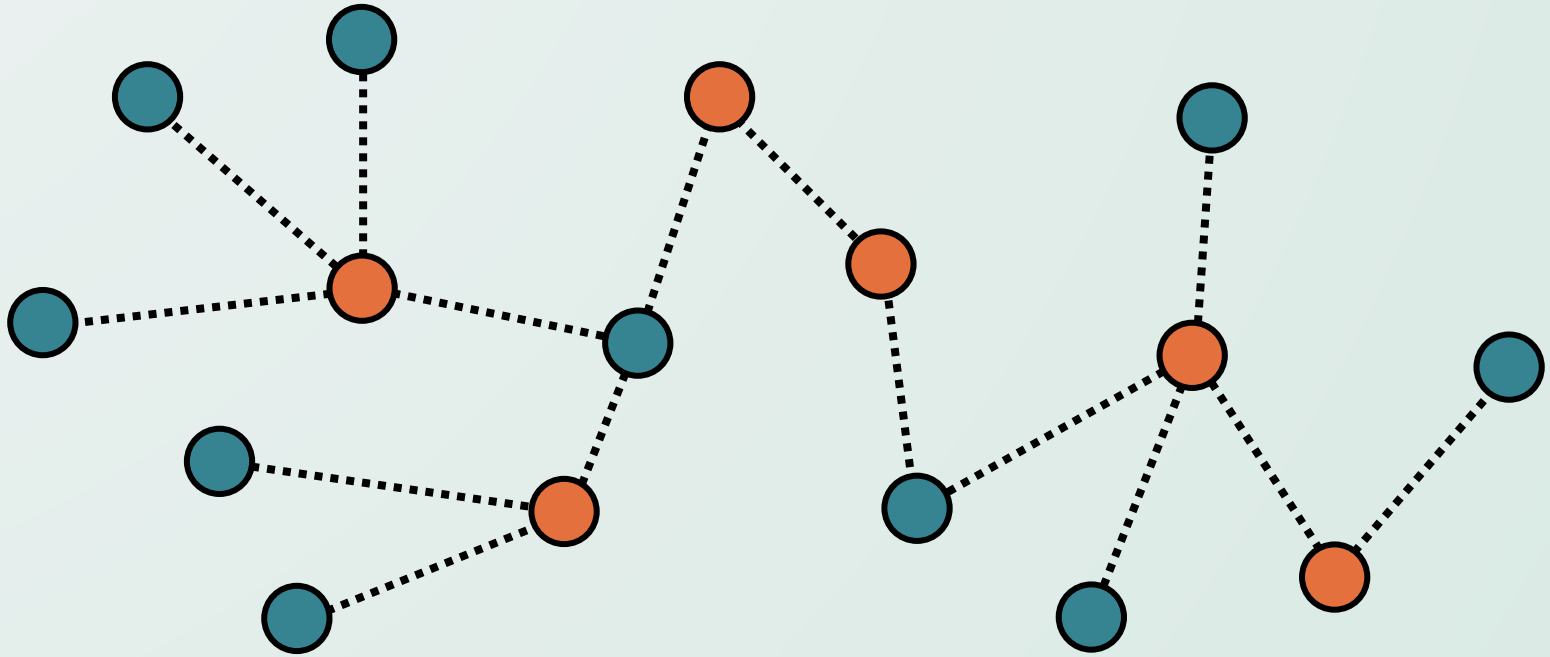
# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



# הפתרון

- נסרוק את כל הצמתים בגרף בזמן לינארי. לכל צומת, נשמור את הדרגה שלה, ונתחזק רשימה של כל העלים (צמתים עם דרגה 1) ביער.
- כל עוד רשימת העלים לא ריקה, נוציא בכל פעם צומת $v$ מהרשימה, נוסיף את השכן היחיד שלה $u$ לכיסוי, ונמחק את $v,u$ ואת כל הקשתות היוצאות מ־$u$.



# הוכחת נכונות

- נוכיח תחילה כי הכיסוי שקיבלנו חוקי.
  - כל עוד יש קשתות בגרף, קיים עלה, ולכן האלגוריתם מסתיים כאשר אין קשתות בגרף.
  - אנו מסירים קשתות מהגרף רק לאחר שאנחנו מכסים אותם. לכן כל קשת מכוסה כנדרש.
- ניתן להוכיח כי הכיסוי שקיבלנו הינו מינימלי ע"י **טיעון החלפה**
  - נשווה את הכיסוי החמדני $A$ לכיסוי אופטימלי אחר $OPT$... כיצד ניתן להשוות בין הכיסויים?

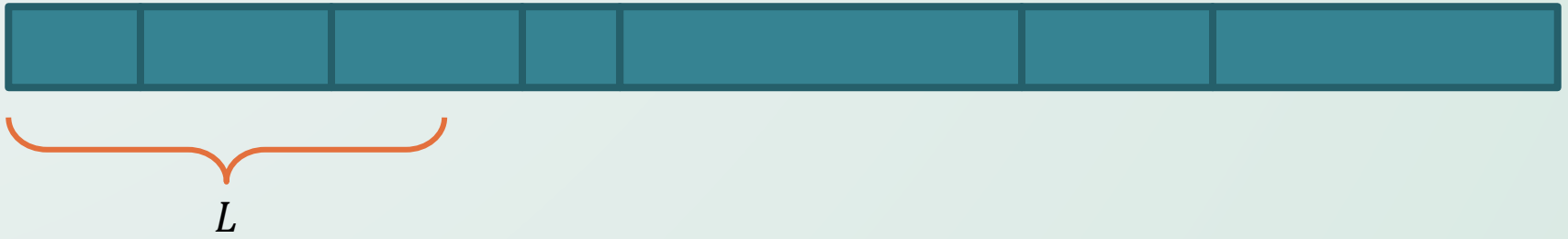
# הוכחת נכונות

- נוכיח תחילה כי הכיסוי שקיבלנו חוקי.
  - כל עוד יש קשתות בגרף, קיים עלה, ולכן האלגוריתם מסתיים כאשר אין קשתות בגרף.
  - אנו מסירים קשתות מהגרף רק לאחר שאנחנו מכסים אותם. לכן כל קשת מכוסה כנדרש.
- ניתן להוכיח כי הכיסוי שקיבלנו הינו מינימלי ע"י **טיעון החלפה**
  - נשווה את הכיסוי החמדני $A$ לכיסוי אופטימלי אחר $OPT$... כיצד ניתן להשוות בין הכיסויים?
- ניתן גם להוכיח כי הכיסוי שקיבלנו הינו מינימלי ע"י **טיעון השוואה**
  - נתבונן על עלה כלשהו $v$ ועל הצומת שאליה הוא מחובר $u$.
  - לפחות אחת מהן חייבת להיות מכוסה. מה יקרה אם נבחר את $v$ לכיסוי?
  - נרצה להוכיח כי תמיד משתלם יותר לבחור דווקא את $u$ לכיסוי.
- **תרגיל לבית:** השלימו את הוכחות הנכונות, בשתי הדרכים השונות.

# בעיית הנגר

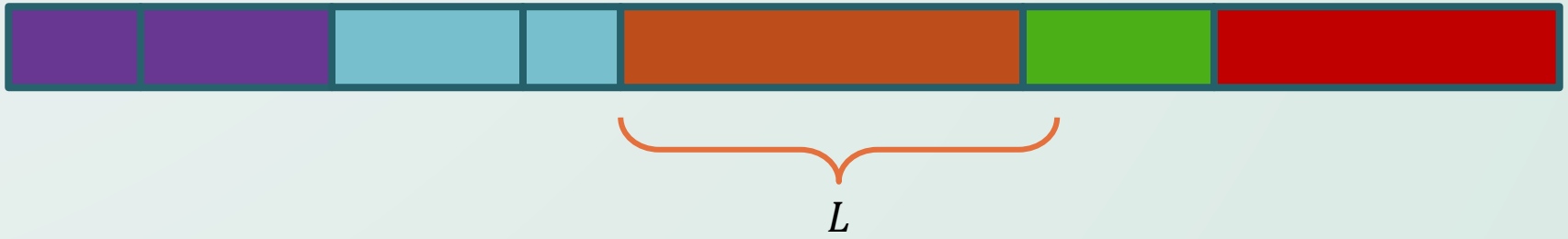
# בעיית הנגר

- נגר מקבל קורת עץ לחיתוך באורך כולל $S$.
- על הקורה מסומנים $n$ מיקומים אפשריים לחיתוך $x_1, x_2, \ldots, x_n$, שרק בהם יכול הנגר לחתוך את הקורה.
- מזמין העבודה זקוק לקורות באורך לכל היותר $L$. בפרט, הוא רוצה שלאחר החיתוך, כל חלקי הקורות יהיו לא יותר ארוכות מ־$L$.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שיבחר את מקומות החיתוך, כך שמספר הקורות באורך לכל היותר $L$ יהיה מינימלי.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



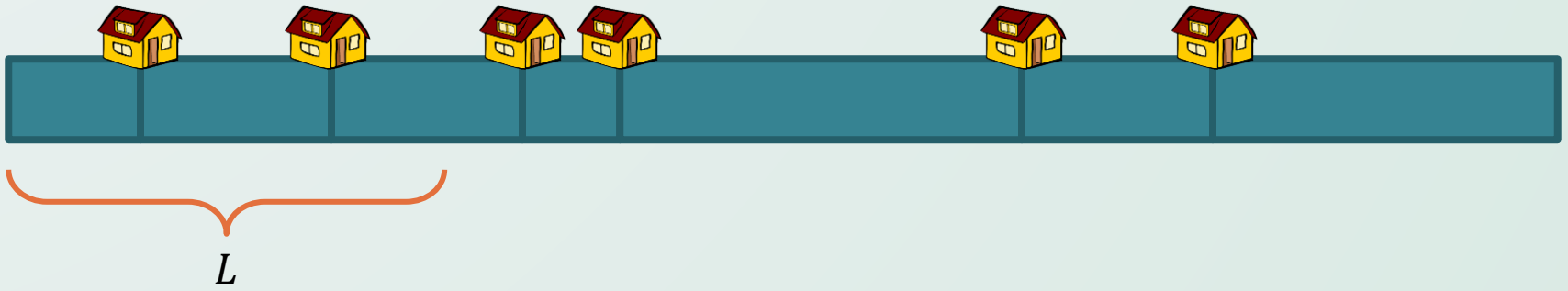
# בעיית הנגר

- נגר מקבל קורת עץ לחיתוך באורך כולל $S$.
- על הקורה מסומנים $n$ מיקומים אפשריים לחיתוך $x_1, x_2, \ldots, x_n$, שרק בהם יכול הנגר לחתוך את הקורה.
- מזמין העבודה זקוק לקורות באורך לכל היותר $L$. בפרט, הוא רוצה שלאחר החיתוך, כל חלקי הקורות יהיו לא יותר ארוכות מ־$L$.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שיבחר את מקומות החיתוך, כך שמספר הקורות באורך לכל היותר $L$ יהיה מינימלי.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



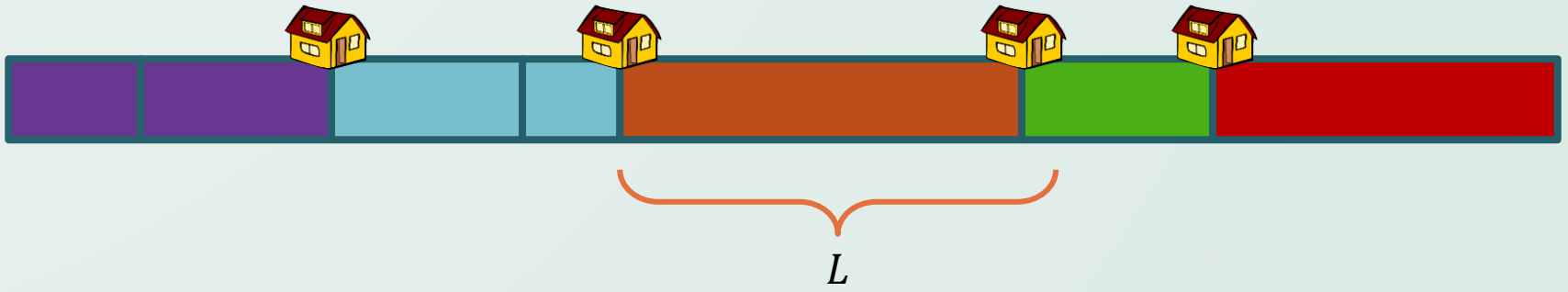
# סיפור אחר, אותה הבעיה

- ישנו מסלול באורך $S$ ועליו מסומנות $n$ נקודות עצירה שונות $x_1, x_2, \ldots, x_n$.
- אתם יודעים כי בכל יום תוכלו ללכת מרחק של לכל היותר $L$. עליכם כעת לבחור את מקומות הלינה, מבין $n$ נקודות העצירה השונות.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שיבחר את מקומות העצירה, כך שתסיימו את המסלול במספר ימים מינימלי.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



# סיפור אחר, אותה הבעיה

- ישנו מסלול באורך $S$ ועליו מסומנות $n$ נקודות עצירה שונות $x_1, x_2, \ldots, x_n$.
- אתם יודעים כי בכל יום תוכלו ללכת מרחק של לכל היותר $L$. עליכם כעת לבחור את מקומות הלינה, מבין $n$ נקודות העצירה השונות.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, **שיבחר את מקומות העצירה, כך שתסיימו את המסלול במספר ימים מינימלי.** הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



# הפתרון

- נעבור על המקומות המסומנים לפי הסדר.
- בכל שלב של האלגוריתם, נבחר לחתוך את הקורה במקום המסומן הרחוק ביותר שאינו רחוק יותר מ־$L$.
  - או, באופן שקול, בכל יום נבחר ללכת עד לתחנת העצירה הרחוקה ביותר שאליה אנחנו יכולים להגיע.
- אך האם זהו הפתרון האופטימלי?
  - אולי, בחלק מהימים, שווה לנו לעצור לא בתחנת עצירה הרחוקה ביותר, כדי שביום הבא נוכל להגיע רחוק יותר?

# הוכחת נכונות

- נסמן את הפתרון החמדני בתור $A = \{x_{i_1}, x_{i_2}, \dots, x_{i_k}\}$. מספר החיתוכים בו הינו $k$.
- נניח כי ישנו פתרון אופטימלי $OPT = \{x_{j_1}, x_{j_2}, \dots, x_{j_m}\}$, שיותר טוב מהפתרון החמדני, כלומר $m < k$.
- נשים לב כי בדומה לטיעון בבעיית השיבוץ בשולחן בודד, מתקיים $x_{i_r} \geq x_{j_r}$ לכל $r \leq m$.
  - במילים, החיתוך ה־$r$ בפתרון החמדני לא יבוא לפני החיתוך ה־$r$ בפתרון האופטימלי (ובעצם, בכל פתרון חוקי).
  - תרגיל טוב לבית! הוכחה באינדוקציה על $r$. 😉
- בפרט מתקיים $x_{i_m} \geq x_{j_m}$, כלומר החיתוך האחרון בפתרון האופטימלי לא יבוא אחרי אותו החיתוך בפתרון החמדני.
- מכיוון ש־$m$ הוא לא החיתוך האחרון בפתרון החמדני, יש עוד לפחות $L$ יחידות אחריו. כלומר בעצם מתקיים $x_{i_m} < S - L$ .
- סה"כ קיבלנו $x_{j_m} < S - L$. אך זהו החיתוך האחרון בפתרון האופטימלי, ולכן חייב להיות במרחק של לכל היותר $L$ מנקודת הסיום $S$. כלומר, חיתוך זה לא חוקי – סתירה!